The Econstellar Compute Engine

A Self-Verifying, Sandboxed Substrate for Reproducible Financial Econometrics, and the Research Programme It Carries

Avishek Bhandari
School of Humanities, Social Sciences and Management
Indian Institute of Technology Bhubaneswar
avishekb@iitbbs.ac.in

June 15, 2026

Abstract

Financial econometrics has a reproducibility problem that is, for the most part, a software problem. Published results depend on undocumented data revisions, on estimator details buried in analysis scripts, and on machine-specific numerics that move the third decimal place. We describe Econstellar, a research programme organised around a single compute engine that treats these as first-class engineering concerns. The engine exposes a parameterised-only registry of twenty-six methods spanning long-memory estimation, wavelet decomposition, transfer entropy, surrogate testing, community detection, network connectedness, and structural channel attribution. Every method ships with a pre-registered, failable test of its own output, and a method is not in the catalogue until it does. A nightly evaluation suite runs the methods on real market data against fixed numerical bands, maintains a machine-written state ledger, and refreshes a public claims record without ever editing a result by hand.

The engine runs across three planes: a scale-to-zero kernel on managed cloud infrastructure, an always-on workstation tower that carries the heavy asynchronous estimators, and static public surfaces that render only what the suite has verified. We show that a governed core budget with pinned linear-algebra threading makes results independent of worker count, and that a GPU port of the k-nearest-neighbour transfer-entropy estimator reproduces the CPU value to within machine epsilon while running several times faster. Six published frameworks, on adaptive market networks, scale-ordered contagion, contagion channels, wavelet quantile transfer, commodity systems, and protocol-mediated systemic risk, are elements of this one programme rather than separate codebases. We report honest negatives alongside positive findings, and we argue that the engineering discipline is itself a contribution.

Contents

1 Introduction **3**
1.1 Thesis: one engine, one substrate, one gate 3
1.2 Engine, not library 4
1.3 Discipline borrowed from systems practice, applied to econometrics 4
1.4 Contributions 5
1.5 Related work 5
1.6 Roadmap 5

2 System Architecture **6**
2.1 The kernel: synchronous methods on scale-to-zero Cloud Run 6
2.2 The tower: heavy asynchronous estimators on an always-on workstation 7
2.3 The surfaces: render only what is verified 8
2.4 The request lifecycle 8

2.5 The registry as the primary safety property . . . 9
2.6 The sandbox, stated precisely . . . 10
2.7 The JSON contract and the purity of stdout . . . 10
2.8 The data plane . . . 11

3 The methodological substrate and the method registry 11
3.1 The eight primitives . . . 11
3.2 The registry and its six families . . . 14
3.3 The transfer-entropy estimator in full . . . 14
3.4 Determinism and reproducibility of the estimators . . . 16
3.5 A failable test per method . . . 17

4 Heterogeneous compute: core governance, GPU offload, and the asynchronous tower 17
4.1 Core governance: one budget, wired everywhere . . . 18
4.2 GPU offload of the transfer-entropy search . . . 19
4.3 The asynchronous tower . . . 20

5 The AI layer: a natural-language client, not a privileged path 21
5.1 The chat route . . . 21
5.2 The research route . . . 22
5.3 The safety argument . . . 22
5.4 Rate limits and the cost bound . . . 23

6 Verification and Epistemics 23
6.1 The evaluation suite . . . 23
6.2 The nightly self-learning loop . . . 25
6.3 The learning ledger . . . 26
6.4 The claims ledger . . . 27
6.5 The formal layer . . . 28

7 The Elements 29
7.1 NAMH: adaptive market networks . . . 29
7.2 SOCH: scale-ordered contagion . . . 30
7.3 Contagion channels . . . 30
7.4 WaveQTE: wavelet quantile transfer . . . 31
7.5 Commodity systems . . . 31
7.6 MCPFM: protocol-mediated systemic risk . . . 32
7.7 Cross-framework consistency . . . 32

8 Deployment and Operations 33
8.1 Deploying the kernel to Cloud Run . . . 33
8.2 Isolation and the Tier-0 hardening posture . . . 34
8.3 The cost bound on the AI layer . . . 34
8.4 The governed cloud-capability layer . . . 34
8.5 The SRI live feed as a standing operation . . . 36
8.6 The nightly self-check loop . . . 36

9 Worked Examples 36
9.1 A synchronous request and its response . . . 36
9.2 An exact reproduction . . . 37
9.3 A GPU re-run, bit-exact to the CPU estimator . . . 38
9.4 A nightly tick of the systemic-risk feed . . . 38

10 Design principles, limitations, and outlook **39**
10.1 The design principles, distilled 39
10.2 Limitations 40
10.3 Towards a self-renewing programme 40

A The method registry **41**

B API reference **43**

C The learning ledger **44**

D Verified results and provenance **45**

1 Introduction

There is a quiet asymmetry in applied financial econometrics. Producing a result is cheap; making that result re-runnable, by someone else, on demand, is expensive, and so it is mostly not done. A reader of a typical spillover or contagion paper is handed a table and an appendix, not a button. When the printed coefficient is later questioned, the burden of reconstruction falls on whoever raises the question, and the original computation, with its package versions, its random seeds, and its data vintage, is rarely recoverable in full.

We take the view that this is, for the most part, a software problem, and that it is worth attacking with the engineering discipline that a serious systems group would bring to any service whose correctness matters. Three failure modes recur. The first is undocumented data revisions: a vendor restates a close, a roll convention shifts a futures series, and a number that was correct last quarter no longer reproduces, with nothing in the published record to say so. The second is estimator detail buried in analysis scripts: an embedding lag, a neighbour count, a bandwidth, a partition of the sample into episodes, choices that move a headline figure but live only in a working directory. The third is machine-specific numerics: an unpinned multi-threaded linear-algebra library, a nearest-neighbour tie broken differently on a different processor, a sum accumulated in a different order, each capable of moving the third decimal place in a way no reader can anticipate. None of these is exotic. Together they are why a result in this field is so often easy to publish and hard to re-run.

1.1 Thesis: one engine, one substrate, one gate

This manuscript describes our response, which is deliberately a programme and not a paper. Econstellar is organised as a research "space programme": one sandboxed compute engine, exposing one methodological substrate of eight primitives, under one pre-registered and failable evaluation gate. The six published frameworks that the group has produced, on adaptive market networks (NAMH), scale-ordered contagion (SOCH), cross-border contagion channels, wavelet quantile transfer (WaveQTE), commodity systems, and protocol-mediated systemic risk (MCPFM), are *elements* of that one programme, not separate codebases that happen to share an author. The same long-memory estimator, the same wavelet decomposition, the same transfer-entropy kernel, the same surrogate machinery, and the same community detection thread through all of them. Verifying the substrate, once, carefully, verifies every element built on it.

The engine itself is small. It runs a fixed catalogue of econometric methods inside a sandbox and returns each result as structured data, with enough provenance attached that the number can be banded against an expectation and reproduced later. As of the live release it serves twenty-six methods, and the most recent full suite recorded twenty-six of twenty-six passing. It is written in Node and R, deployed for the research platform of the School of Humanities, Social

Sciences and Management at IIT Bhubaneswar, and documented from the engine outward in the published reference [6]. The design thesis is one sentence: *a method is not in the catalogue until it ships with a failable test of its own output.* The catalogue and that suite of tests are the deliverable; the elements are what the deliverable certifies.

1.2 Engine, not library

It would be natural to read what follows as a library of econometric routines. It is not, and the distinction is load-bearing both for safety and for trust.

A library accepts arbitrary code and runs it; the surface for arbitrary-code execution is the point. The engine accepts no code. A request names a method from a fixed registry and a parameter object validated against a schema, and the runner scripts that do the work are part of the repository, never user-supplied. **The parameterised-only registry is therefore the primary guard against arbitrary-code execution**, and the operating-system sandbox is defence in depth on top of it, not the load-bearing control. On the workstation and in local development each analysis runs under `bubblewrap` with the network unshared, the filesystem read-only, and a fresh scratch space; the deployed cloud image ships without that tool and runs under a wall-clock timeout with the managed runtime as its kernel boundary. Either way, no user input reaches a shell, because the registry admits no user code in the first place. We return to this topology, and to the three planes the engine runs across, in Section 2.

The other half of "engine, not library" is the catalogue rule already stated: a method joins only with a pre-registered, failable test of its own output, and holes in coverage are marked rather than filled. The registered voice of the project is blunt about the trade this implies. *We would rather have twenty-six checked methods than fifty asserted ones.* A figure that the engine reports is a figure the engine can fail to reproduce, loudly and in public, and that is the property we are trying to buy.

1.3 Discipline borrowed from systems practice, applied to econometrics

The engineering posture here is borrowed, openly, from how top-tier systems and research-infrastructure groups treat reliability and verification: correctness asserted by a pre-registered evaluation rather than by the authors, every figure resolving to a job identifier or a revision string, machine-written ledgers that are never hand-edited, and honest reporting of what does not work. What is unusual is the domain. We apply that discipline to financial econometrics, a field whose objects are noisy, whose data is revised, and whose estimators are sensitive to choices that rarely survive into print. The combination is the contribution: not any single number, but the apparatus that makes each number re-runnable and the negatives as visible as the positives.

That apparatus is needed precisely because of what the markets are. Under the adaptive markets hypothesis [28], efficiency is not a fixed property but an evolutionary one: heterogeneous participants adapt at different rates, relationships strengthen and decay, and the structure a study measures in one regime need not hold in the next. The Grossman–Stiglitz argument [22] sharpens the point from the side of information: if prices revealed all private information costlessly, no one would pay to gather it, so informationally efficient markets are impossible and a residue of private, costly, unequally held information must remain. Markets that are adaptive, heterogeneous, and information-driven in this sense do not yield to a single estimate stated once. They demand a measurement apparatus that can be re-run as the regime turns, that states its full parameterisation, and that reports honestly when a structure it found before has dissolved. The engine is our attempt to build exactly that apparatus and to hold it to a standard a reader can check.

1.4 Contributions

This manuscript makes the following contributions.

1. **A programme framing.** We argue that six separately published frameworks are better understood, and better maintained, as elements of one research programme sharing a single eight-primitive substrate and a single evaluation gate, and we make that claim concrete by exhibiting which primitives each element instantiates (Section 7).

2. **An engine whose safety rests on parameterisation.** We describe a compute engine in which the parameterised-only registry, not the sandbox, is the primary control against arbitrary-code execution, with the sandbox as defence in depth (Section 2).

3. **A catalogue under a failable gate.** We state and apply the rule that a method enters the catalogue only with a pre-registered, failable test of its own output, served live as twenty-six methods at twenty-six of twenty-six passing, and we describe the evaluation discipline that distinguishes a band failure from a harness failure (Section 7 and the verification section).

4. **Governed determinism and bit-exact acceleration.** We show that a governed core budget with pinned linear-algebra threading makes a result independent of worker count, and that a GPU port of the transfer-entropy nearest-neighbour search reproduces the CPU estimate to within machine epsilon while running several times faster (the heterogeneous-compute section).

5. **Honest negatives as first-class results.** We report the negatives alongside the positives, including a contagion significance test that is not significant, a false-discovery-controlled network that is degenerate, and a pre-registered model re-evaluation whose discrimination falls below chance, and we treat each as an immutable fact of the record (Sections 7 and 10).

1.5 Related work

Three literatures meet in this engine. The first is the long concern with reproducibility in empirical economics, opened by the *Journal of Money, Credit and Banking* replication project [15] and sharpened by studies of the numerical reliability of econometric software [30] and of reproducible econometric practice [24]. The computational-science community has codified much of the operational discipline we adopt: simple rules for reproducible research [32], container-based environments [12], and the older idea of literate, provenance-carrying programs [23]. Our addition to this lineage is not another guideline but a running engine in which the discipline is enforced by a failable gate.

The second is the information-theoretic and connectedness toolkit the substrate implements. Transfer entropy [33] and its nearest-neighbour estimation [21, 25] underpin the engine's directional-flow methods; effective transfer entropy and surrogate correction [29, 35] control bias, applied financial studies [19] motivate the design, and detrended fluctuation analysis [31] is the long-memory primitive. The connectedness family follows the Diebold–Yilmaz programme [16–18] and its frequency extension [3], and we read the systemic-risk feed alongside the standard market measures: CoVaR [2], systemic expected shortfall [1], and SRISK [14].

The third is the view of market efficiency the elements speak to: the efficient-markets benchmark [20], the impossibility of informationally efficient markets [22], heterogeneous-agent dynamics [13], and the adaptive-markets synthesis [28] that gives the NAMH element its name.

1.6 Roadmap

The paper proceeds from the engine outward. Section 2 sets out the three-plane architecture, the request lifecycle, and the sandbox topology. The substrate section catalogues the eight

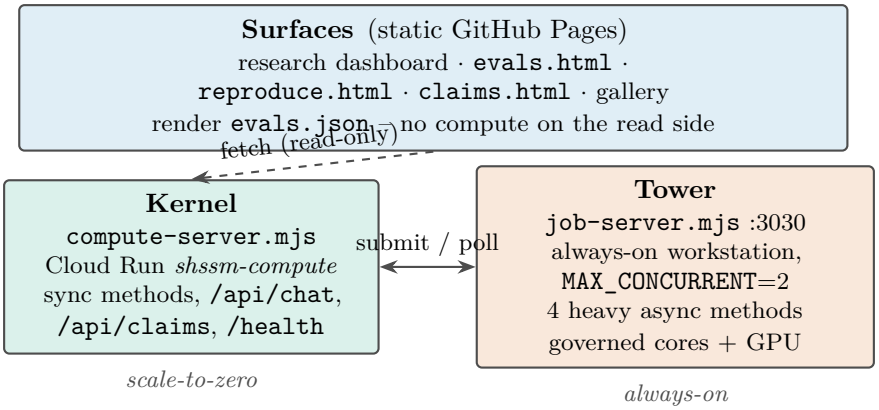

Figure 1: The three planes. The read side fetches but never computes; the kernel serves synchronous methods and submits heavy work to the tower.

primitives and the twenty-six methods built on them, and the heterogeneous-compute and AI-layer sections cover the governed core budget, the GPU offload held to a bit-exact bar, and the natural-language front door onto the same validated registry. The verification and epistemics section states how a pre-registered, failable evaluation suite, a machine-written state ledger, and a claims record that never deletes combine into a self-checking system, with worked examples following. Section 7, *The Elements*, then reads the six published frameworks as elements of the one programme, each with its economic question, the substrate primitives it instantiates, its headline verified result, and its honest status. Section 10 distils the design principles, states the limitations plainly, and describes the self-renewing programme we are building towards.

2 System Architecture

The engine has one job: run a fixed catalogue of econometric methods, return each result as structured data, and attach enough provenance that the number can be banded against an expectation and reproduced later. Everything in this section follows from a single decision – that a caller may name a method and supply parameters, but may never supply code. That decision is what makes the engine safe to expose publicly, and it is what lets the same method run identically in a cloud container, on the laboratory workstation, and in the test harness.

We organise the system into three planes, each chosen for a different operational regime. The *kernel* answers synchronous requests on scale-to-zero Cloud Run, which suits bursty, low-volume research traffic at near-zero idle cost. The *tower* carries the heavy asynchronous estimators that a request-scoped cloud service cannot, on an always-on workstation with governed cores and a GPU. The *surfaces* are static pages that render only what the evaluation suite has verified, so the read side computes nothing and can never present an unchecked figure. Figure 1 sketches the topology.

2.1 The kernel: synchronous methods on scale-to-zero Cloud Run

The kernel is `server/compute-server.mjs`, a zero-dependency Node service that needs only `node:http`, `node:https`, `node:child_process` and `node:fs`. It is deployed to Google Cloud Run as the service `shssm-compute` in region `asia-south1`; the live release is revision `shssm-compute-00036-llf`, re-confirmed on 2026-06-15 serving a 26-method registry. A liveness probe reports the running configuration directly: `GET /health` returns `{ok:true, methods:26, revision:"shssm-compute-00036-llf", sandbox:"timeout", timeout_s:90}`. The same code runs locally on port `3200` for development.

Table 1: The kernel HTTP surface (`server/compute-server.mjs`). Read-side pages only ever consume these routes; the per-minute and per-day caps are the abuse and credit guards of Section 8.

Route	Method	Purpose	Cap
`/api/compute/run`	POST	run one registry method on validated params	20/min
`/api/compute/catalog`	GET	method + dataset registry the workbench binds to	–
`/api/chat`	POST	AI analyst (Gemini function-calling over registry)	10/min, 50/day
`/api/research`	POST	deep-research assistant (agentic + grounded)	5/min, 20/day
`/api/situate`	POST	place a result against the verified record	30/min
`/api/claims`	GET	epistemic ledger (established / contested / superseded)	–
`/api/sri/current`	GET	systemic-risk live-feed: latest value	–
`/api/sri/history`	GET	systemic-risk live-feed: time series	–
`/api/sri/network`	GET	systemic-risk live-feed: directed edges	–
`/api/sri/cron-tick`	POST	the daily SRI append (Cloud Scheduler only)	–
`/api/event`	POST	read-side telemetry (aggregate counts, no PII)	60/min
`/health`	GET	liveness: methods, revision, sandbox	–
`/metrics`	GET	request / cache / rate-limit counters	–

Cloud Run scales the kernel to zero between requests. For a research service whose traffic is occasional and bursty – a workbench query here, a chatbot turn there – this is the right trade: we pay for compute only while a request is in flight, and a cold start is acceptable latency for an interactive analysis. The cost of that choice is the request budget. Cloud Run bounds a request at 300 s and may throttle CPU outside an active request, so any estimator whose honest runtime is measured in minutes cannot live on the synchronous path. Those estimators are the tower's (Section 2.2); the kernel carries the rest, which return in well under the timeout.

The kernel's public surface is a small, rate-limited JSON API. CORS is open so the static read-side pages can fetch it, but those pages only ever read; nothing on the read side mutates engine state. Table 1 reproduces the routes. The two model-backed routes, `/api/chat` and `/api/research`, share a per-instance daily budget and, over cap, return `503 daily_capacity` with a reset hint rather than a fabricated answer; that budget and its cost ceiling are treated in Section 8.

2.2 The tower: heavy asynchronous estimators on an always-on workstation

Four methods cannot live on the synchronous path, because their honest runtimes exceed what a scale-to-zero, request-scoped service can carry: ksg_te (exact KSG/Frenzel–Pompe transfer entropy with IAAFT surrogates across all directed pairs), ksg_robustness (its nuisance-grid sweep), namh_pipeline (the end-to-end NAMH estimation, roughly 24 minutes at $B = 200$), and soch_robustness (a bootstrap symmetry-test grid). These run on the tower, `server/job-server.mjs`, an always-on service on port `3030` on the group's 22-core workstation, which also carries the GPU used to offload the transfer-entropy search (treated in the heterogeneous-compute section).

The tower inverts the kernel's request model. `POST /api/jobs/submit` validates the method and returns a `job_<date>_<hash>` id immediately with `202 Accepted`; the work then runs asynchronously at a small fixed concurrency (`MAX_CONCURRENT = 2`). A caller polls `GET /api/jobs/{id}` for the record, or subscribes to `GET /api/jobs/{id}/stream`, a Server-Sent-Events channel that emits `progress` events and terminates on `done` or `error`. The progress events are exactly the `CE_PROGRESS`-gated lines of Section 2.7: the tower spawns the same runner with `CE_PROGRESS=1`, parses each newline-delimited progress object to update the job, and keeps the final unterminated object as the result.

Job state persists to disk. Every job record is written to `.jobs/` as a JSON file on each transition, so the tower survives a restart: on boot it re-reads `.jobs/`, and any job left `running` or `queued` is re-queued rather than lost. Crucially, the tower is also where core governance lives – it sets a hard per-job core ceiling and pins the numerical libraries to a single thread each before spawning R, so two concurrent jobs cannot oversubscribe the machine. That mechanism, and the 22-core saturation it fixed, are the subject of the heterogeneous-compute section; here it suffices that the budget is set by the plane that owns concurrency.

2.3 The surfaces: render only what is verified

The third plane is the read side: static GitHub Pages in the sibling `econstellar/` checkout – the research dashboard, the evaluation and reproduction pages, the claims panel, the gallery. Nothing on the read side computes. Each page fetches and renders `evals.json`, the single machine-written artefact of one genuine run of the public evaluation suite, and the kernel's read-only routes (`/api/claims`, the SRI feed). A figure appears on a surface only because the suite produced it under a pre-registered, failable band; a regression turns the dashboard red rather than passing silently.

Principle 1 (Render only what is verified)**.** The read surfaces display `evals.json` and the kernel's read routes; they execute no analysis of their own. A number reaches a reader only after the evaluation suite has produced it and checked it against a band fixed before the run. The read side cannot present an unchecked figure because it cannot compute one.

This is the architectural counterpart of the catalogue discipline. The kernel will not run a method that is not registered; the surfaces will not show a result that the suite has not verified. The separation also keeps the planes honest about cost and blast radius: the read side is free static hosting that can fail closed, the kernel pays only while computing, and the heavy machine is reached only through a job submission that the kernel itself gates.

2.4 The request lifecycle

Every synchronous analysis follows one path, and the path is the same whether the caller is the workbench, the chatbot, or the test harness. We trace it concretely for `POST /api/compute/run`.

1. **Receive.** The body is read with a 64 KB cap; an oversized body is rejected with `413` before parsing, so no request can fan out into unbounded memory. A malformed body returns `400 bad JSON body`.
2. **Validate.** `validate(method, params)` looks the method up in the fixed registry. An unknown name returns `400 unknown method`. Each declared parameter is then coerced and checked against its typed schema (Section 2.5); a market name not in the panel's whitelist is rejected as `unknown series`. What reaches R is a *clean* parameter object, never the raw request.
3. **Route by kind.** If the method is flagged `long_running` it is refused here with `{error:"...background job...", async:true}` and directed to the tower; the synchronous endpoint never spawns a heavy estimator.
4. **Cache.** A 5-minute response cache, keyed on method and cleaned parameters, short-circuits repeated identical analyses. Errors are never cached.
5. **Spawn.** On a cache miss the kernel builds the runner argument vector and spawns the registered script under the sandbox (Section 2.6): `Rscript r/<script>.R '<params-json>'`, wrapped in `timeout`. The R helper reads exactly the whitelisted dataset and computes.

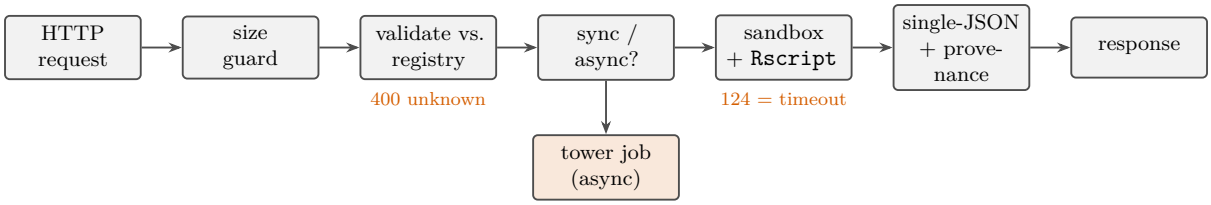

Figure 2: The synchronous request lifecycle. Validation against the registry precedes any spawn; a method flagged long-running is refused synchronous service and submitted to the tower; a wall-clock timeout surfaces as an explicit error, never a silent hang.

6. **Collect.** The child's `stdout` is accumulated and parsed as a single JSON object (Section 2.7). A `124` exit is surfaced as an explicit `timeout after Ns` error, never a silent hang; a non-JSON stdout is reported as such rather than passed through.
7. **Respond.** The result is wrapped with a provenance stamp – method version, engine revision, cleaned parameters, panel id and byte-hash, timestamp, and a permalink that re-runs the exact analysis on load – and returned.

Listing 1 states the same path as pseudocode, to make the ordering of the guards unambiguous: validation precedes any spawn, and the registry lookup is the only thing that turns a request name into an executable.

```
// POST /api/compute/run (server/compute-server.mjs)
if (bodyBytes > 64*1024) return send(413); // size guard
let payload = JSON.parse(body); // 400 on bad JSON
let { method, clean } = validate(payload.method, // 400 unknown method
                                 payload.params); // typed-param coercion
if (method.long_running) return send(400, async); // -> tower, never here
let hit = cacheGet(method, clean); // 5-min TTL
let result = hit
  || await runSandboxed(method, buildArgsJson(method, clean));
return send(200, { ...result,
                   provenance: stamp(method, clean) }); // version+revision+hash
```

Listing 1: The synchronous request path, in brief. Validation and registry lookup precede every spawn; the runner receives a cleaned parameter object, never caller-supplied code.

2.5 The registry as the primary safety property

The central design fact is that the registry is parameterised only. A method entry is a fixed runner script plus a typed parameter schema – for example, the stationarity method binds `r/adf.R` with a `series` parameter that must be exactly one market drawn from the panel's whitelist. The runner scripts in `r/` are part of the repository and are never user-supplied. A caller therefore chooses *which* registered analysis to run and supplies typed inputs to it, but has no channel through which to inject code, a file path, or a shell fragment.

Invariant 1 (Parameterised-only execution)**.** The only executables the engine ever runs are the registered runner scripts shipped in the repository. A request supplies a method name from a closed registry and a parameter object validated against that method's typed schema. No request input is interpreted as code, a path, or a command. There is therefore no arbitrary-code-execution surface, independent of any sandbox.

This is the load-bearing claim, and it is worth being precise about why. The validation step (Section 2.4) coerces each parameter to its declared type and rejects anything outside its domain: series names are checked against the dataset whitelist, integers and numbers must parse finitely,

enumerations must match a fixed value set, and counts must fall in the method's declared arity. A string that is not a registered method name does not select a default; it is refused. A probe such as `"DROP_TABLE; rm -rf /"` as a method name returns `400 unknown method` and reaches no shell. The safety of the public endpoint rests here, on Invariant 1, not on the sandbox.

The sandbox is defence in depth, and we are careful not to overstate it (Section 2.6). Stating the property in the order it actually holds – registry first, sandbox second – matters because the two deployments run different sandbox modes, and the safety argument must not depend on which one is live.

2.6 The sandbox, stated precisely

The runner is launched by one function, `runSandboxed`, which detects whether `bwrap` (bubblewrap) is present and adapts.

On the workstation and in local development, where `bwrap` is installed, each analysis runs under bubblewrap with the whole filesystem read-only (`--ro-bind / /`), no network (`--unshare-net`, verified to block outbound), a fresh `tmpfs` scratch, and `--die-with-parent`, the whole wrapped in `timeout`. The deployed Cloud Run image, by contrast, ships *without* `bwrap`. There the kernel runs in `timeout`-only mode – which is exactly what `/health` reports as `sandbox:"timeout"` – and the isolation boundary is gVisor, the sandbox Cloud Run already places around the container, together with the read-only image and Cloud Run's own egress posture. We do *not* claim that `bwrap` runs in production; it does not. On Cloud Run, the kernel boundary is gVisor and a wall-clock timeout bounds each call, while the no-arbitrary-code guarantee comes from Invariant 1, which holds identically in both modes.

Principle 2 (Defence in depth, not in place of the registry)**.** The sandbox narrows what a misbehaving runner could touch; it is not the reason the public endpoint is safe to expose. That reason is the parameterised-only registry. Where `bwrap` is present we additionally pin the filesystem read-only and the network off; where it is not, gVisor and a per-call timeout provide the kernel boundary. The safety claim is invariant to which mode is live.

Either way, one method call is bounded by `COMPUTE_TIMEOUT_S` – 90 s in the deployed image, default 60 s – and a timeout is surfaced as an explicit error, so a pathological input degrades to a clean `timeout after Ns`, never an unbounded run.

2.7 The JSON contract and the purity of stdout

The data contract is deliberately minimal: one JSON object in, one JSON object out. A request carries `{method, params}`; a runner prints exactly one JSON object to `stdout` and exits. This single discipline is what lets a method run identically across the three planes – the cloud container, the workstation, and the harness all spawn the same script and parse the same object. The R helpers in `r/_io.R` centralise emission through one function, `ce_emit`, which serialises the result and quits; failures emit `{error: ...}` through `ce_fail`.

The fragility this guards against is real. R routinely writes to `stdout` for reasons unrelated to the result – a warning that a KPSS p-value lies beyond the tabulated range, a package's registration notice on load. Any such stray line would be concatenated with the JSON and break the parse. The helpers therefore set `options(warn = -1)` and suppress load messages, so that the only thing on `stdout` is the result object.

Progress reporting is the interesting case, because the heavy tower estimators genuinely need to stream progress, yet the kernel's stdout must stay pure. The engine resolves this with a single environment gate. The helper `ce_progress` emits a newline-terminated progress line *only* when the async worker has set `CE_PROGRESS=1`; under the synchronous kernel that variable is unset, so `ce_progress` is a silent no-op and the pure-single-JSON contract of `/api/compute/run` is unchanged. The tower (Section 2.2) sets `CE_PROGRESS=1`, reads the newline-delimited progress

lines to drive its event stream, and treats the final unterminated object as the result. The same runner thus serves both planes without a branch in its own logic.

2.8 The data plane

The primary stored panel is `papers/contagion-channels/data/G20.xlsx`: daily equity log-returns for 18 markets spanning 2006-01-12 to 2026-03-18, which yields $18 \times 17 = 306$ directed market pairs for the information-flow methods. Transfer entropy is computed on log-returns, which are $I(0)$, never on price levels, which are $I(1)$; the stationarity gate is enforced rather than assumed. A second panel, `g20_24`, supplies the 24-series set used by the NAMH methods (19 equity indices plus five commodity futures), read from the published `namh` package's bundled data so that the cloud image and the workstation see byte-identical inputs.

Both panels are byte-hashed with SHA-256 at kernel and tower startup, and the hash travels with the result in its provenance stamp. A reported figure therefore names the exact data state it was computed against, which is what makes a later reproduction meaningful: the panel is identified not by a filename but by the hash of its bytes. Datasets that are planned but not provisioned – an intraday G20 panel, for instance – are present in the registry as honest placeholders with no series list, so `validate` refuses any analysis against them rather than fabricating a result. This is the data-plane expression of the same discipline that governs the method catalogue: a capability is exposed only when it is real.

3 The methodological substrate and the method registry

The engine's claim to be a single substrate, rather than a bag of scripts, rests on one observation. The six published frameworks it carries (NAMH, SOCH, contagion-channels, WaveQTE, the commodity study, and MCPFM) do not each need a private toolchain. They are recombinations of a small number of estimator-level operations: measure how a series remembers its own past, split its variance across time scales, quantify directed information flow, build a null by surrogate construction, partition a network into communities, decompose a system's connectedness, attribute a transmitted shock to a structural channel, and validate a binary call against a curve. We call these the *eight primitives*. Every one of the **26** registry methods is one or more primitives wired to a dataset, and every method ships with a pre-registered, failable test of its own output. The registered voice is deliberate: we would rather have twenty-six checked methods than fifty asserted ones.

This section names the eight primitives and their mathematical objects (§3.1), shows how the registry's six families map onto them (§3.2), then gives the transfer-entropy estimator in full because it is the load-bearing one and the only quantity the GPU path must reproduce bit-for-bit (§3.3). We close with the determinism discipline that makes the estimators reproducible (§3.4) and the failable-test rule that governs catalogue entry (§3.5).

3.1 The eight primitives

Table 2 names each primitive, the mathematical object it returns, its canonical reference, and the registry methods that instantiate it. The primitives are estimator-level, not framework-level: a single method (for example namh_pipeline) composes several, and a single primitive (for example P3) is shared across frameworks that otherwise have nothing in common.

P1 – long memory. The object is the Hurst exponent H, the scaling exponent of detrended fluctuation analysis. On the cumulative profile $y_t = \sum_{s \le t}(x_s - \bar{x})$ the series is cut into log-spaced boxes, each box is detrended by a local polynomial, the root-mean-square residual fluctuation $F(n)$ is taken per box length n, and H is the slope of $\log F(n)$ on $\log n$. The transparent

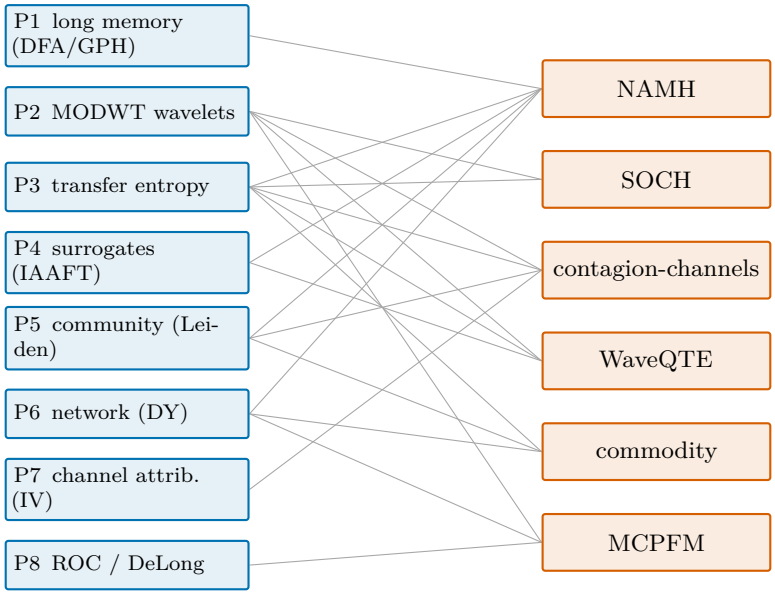

Figure 3: The substrate carries the programme. Each published framework (right, in orange) composes a subset of the eight estimator-level primitives (left, in blue); the same transfer-entropy and network machinery recurs across elements. The composition shown is the principal-primitive view; Section 7 gives each element in full.

realisation (`r/dfa_hurst.R`) implements this directly; the published path (`r/namh_hurst.R`) calls namh::estimate_hurst_panel [9] on a rolling window and supplies the NAMH node weight $\phi(H) = 1 - 2|H - \frac{1}{2}|$, a local-efficiency score peaking at the random-walk value $H = \frac{1}{2}$. We read H on log-returns, which are $I(0)$, never on price levels, which are $I(1)$.

P2 – multi-scale decomposition. The object is the variance of a series across dyadic time scales, obtained from the maximal-overlap discrete wavelet transform (MODWT). Boundary-affected coefficients are removed by the brick.wall operator before the per-scale variance $\sigma^2_{d_j}$ is read, so scale d_1 corresponds to roughly 2–4 day fluctuations, d_2 to 4–8 days, and so on (`r/wavelet.R`). The same decomposition underlies the scale-resolved squared coherence $\rho^2_j = \mathrm{cor}(d^x_j, d^y_j)^2$ (`r/wavelet_coherence.R`), which we label honestly as a discrete scale-resolved coherence rather than the continuous Morlet time-frequency object. P2 is the Stage-1 primitive of NAMH, MCPFM, and the contagion-channels frameworks.

P3 – directed information flow. The object is transfer entropy $T_{Y \to X}$, the reduction in uncertainty about X_{t+1} from knowing the past of Y beyond the past of X [33]. Three estimators sit under this primitive: the engine's own Kraskov–Stögbauer–Grassberger / Frenzel–Pompe k-nearest-neighbour estimator (ksg_te, derived in full in §3.3); the published namh package's Euclidean RANN variant (namh_te); and a transparent wavelet-quantile measure built from MODWT and quantile regression (wqte). These are different estimators of the same object and their magnitudes are not cross-comparable, a boundary the runners state explicitly.

P4 – surrogate construction. The object is a null distribution for a dependence statistic, built without parametric assumptions. The engine's default is the iterative amplitude-adjusted Fourier transform (IAAFT) of Schreiber and Schmitz [34]: a surrogate of the source series is formed by alternately imposing the source's Fourier amplitude spectrum and its empirical amplitude distribution until the rank order stabilises, so the surrogate preserves the source's linear autocorrelation and marginal while destroying its (possibly nonlinear) predictive dependence

Table 2: The eight-primitive substrate. Each primitive is an estimator-level operation with a fixed mathematical object and a canonical reference; the registry methods in the last column instantiate it on a whitelisted dataset.

Primitive	Object computed	Reference	Example methods
P1 long-memory	Hurst exponent H; $\phi(H) = 1 - 2\lvert H - \frac{1}{2}\rvert$	DFA / GPH / Qu	dfa_hurst, namh_hurst
P2 multi-scale	MODWT detail variance $\{\sigma^2_{d_j}\}$	[3]	wavelet, wavelet_coherence
P3 information flow	directed TE $T_{Y \to X}$	[21, 25, 33]	ksg_te, namh_te, wqte
P4 surrogate null	IAAFT / permutation null distribution	[34]	ksg_te, namh_pipeline
P5 community	partition / modularity Q	[36]	network, namh_pipeline
P6 connectedness	GFEVD spillover; frequency bands	[3, 17, 18]	connectedness, spillover_rolling
P7 channel attribution	per-channel share via IV / 2SLS	[10]	channel_attribution
P8 ROC validation	AUC; DeLong / Hansen test	[5]	MCPFM out-of-sample call

on the target. Significance of a directed TE edge is then $p = (1 + \#\{T^{\text{surr}} \geq T^{\text{obs}}\})/(B + 1)$. Surrogate subtraction also defines *effective* transfer entropy (§3.3).

P5 – community detection. The object is a partition of a directed weighted network into communities, with a modularity score Q. The published NAMH pipeline uses the Leiden algorithm [36], which guarantees well-connected communities and corrects the resolution-limit pathologies of Louvain; the transparent network surface uses a Walktrap partition on the undirected projection of a Granger graph. A community result is only as meaningful as the adjacency it partitions: when the upstream FDR gate retains no edges, the partition is degenerate by construction and is reported as such, not dressed up.

P6 – connectedness and network formation. The object is a system's total and directional connectedness from a vector autoregression's generalised forecast-error variance decomposition (GFEVD). The time-domain index is Diebold–Yılmaz [17, 18]; the frequency decomposition is Baruník–Křehlík [3], which splits the total connectedness index across short (≤ 5 day), medium (5–20 day), and long (> 20 day) bands by partitioning the spectral GFEVD at $\omega = 2\pi/P$ (`r/connectedness.R`). The rolling variant (spillover_rolling) tracks the index through time.

P7 – structural channel attribution. The object is the share of a transmitted shock attributable to each of a fixed set of economic channels, identified with instruments rather than read off a reduced-form correlation. The published contagion-channels pipeline [10, 11] runs a two-stage detection-and-attribution procedure: Stage 1 detects directed tail linkages, Stage 2 attributes them to five mutually exclusive channels (trade, financial, geopolitical, behavioural, monetary) by instrumental-variables / two-stage least squares (`r/channel_attribution.R`). This is the primitive that turns a network edge into an economic explanation.

P8 – ROC validation. The object is the area under the receiver-operating-characteristic curve (AUC) of a binary forecast, with a significance test on the AUC. This is the primitive by which a predictive claim is held to account out of sample; MCPFM [5] reports AUC with a DeLong test in version 1 and, under a pre-registered re-evaluation, a Hansen superior-predictive-ability test in version 2. We report both openly (§3.5).

Table 3: The six families of the registry (26 methods total), with a representative method per family, the substrate primitives it composes, and one verified value. The full per-method acceptance bands are in Appendix A. Counts: stationarity 5, volatility 5, information flow 5, connectedness 6, contagion 2, reproduction 3.

Family (n)	**Representative method**	**Primitives**	**Verified value**
Stationarity, cointegration, panels (5)	panel_unit_root, vecm	–	IPS -77.26; Johansen rank 3
Volatility, long memory, spectra (5)	dfa_hurst, wavelet	P1, P2	India $H = 0.542$; $d_1 = 47.07\%$
Information flow (5)	ksg_te, wqte	P3, P4	USA→Japan TE 0.154234
Connectedness, networks (6)	connectedness, network	P5, P6	TCI 30.25%; density 0.5667
Contagion, systemic risk (2)	channel_attribution, sri_daily	P5, P7	GFC trade 27.9%
Reproduction, formal verification (3)	namh_reproduce, soch_robustness	P1–P5	max $\lvert\Delta\rvert$ 1.55×10^{-15}

3.2 The registry and its six families

The **26** methods are grouped into six families by the statistical question they answer, not by which primitives they happen to share; a family is the unit a reader reasons about, while a primitive is the unit the code reuses. Table 3 gives the family structure with a representative method or two from each, its primitive composition, and one verified value. The complete per-method registry with every pre-registered acceptance band is Appendix A; here we show only the shape.

Two points the table cannot show. First, the families are not silos: the same estimator can appear in several. Transfer entropy (P3) is the engine of the information-flow family, but it is also the first stage of the NAMH pipeline in the reproduction family. Second, a method may be either a *transparent* realisation built from substrate parts, or a *published* method that calls a paper's own package directly. wqte reimplements the WaveQTE Stage-1 primitive from waveslim and quantreg because the packaged estimator does not load in the image; soch_profile, channel_attribution, namh_hurst, and namh_te call sochcontagion, contagionchannels, and namh respectively, so the live engine runs the paper's own code. The distinction is recorded on every such method, because a transparent reimplementation and a bit-exact package call carry different reproduction claims.

3.3 The transfer-entropy estimator in full

Transfer entropy is the load-bearing primitive: it is the most-used edge statistic in the programme, the one quantity the GPU path must reproduce bit-for-bit, and the one whose estimator subtleties most repay a careful statement. We give it here at the depth of the implementation in `r/_ksg_core.R` and `gpu/ksg_gpu.py`.

Transfer entropy as conditional mutual information. For an ordered pair $(Y \to X)$ with history length (lag) ℓ, transfer entropy is the conditional mutual information between the one-step future of the target and the past of the source, given the past of the target [33]:

$$T_{Y\to X} \;=\; I\Big(X_{t+1}\,;\, Y_t^{(\ell)} \;\Big|\; X_t^{(\ell)}\Big), \qquad Y_t^{(\ell)} = (Y_t, Y_{t-1}, \ldots, Y_{t-\ell+1}). \tag{1}$$

The conditioning set $Z = X_t^{(\ell)}$ is the target's own past; the marginal blocks are $X = X_{t+1}$ (the future, dimension 1) and $Y = Y_t^{(\ell)}$ (the source past, dimension ℓ). The engine's default is $\ell = 1$ (ksg_te parameter `lag`), the Markov-1 conditioning under which the past blocks collapse to the single lagged values X_t and Y_t. The embedding aligns the future X_{t+1} with the lag blocks ending at t, so that one point of the joint sample is $(X_{t+1}, Y_t^{(\ell)}, X_t^{(\ell)})$.

The KSG / Frenzel–Pompe estimator. Rather than bin the joint density, the estimator counts neighbours. Following Kraskov et al. [25] for mutual information and Frenzel and Pompe [21] for its conditional extension, the conditional mutual information in Eq. (1) is estimated by

$$\widehat{I}(X;Y \mid Z) \;=\; \psi(k) \;+\; \frac{1}{N}\sum_{i=1}^{N}\Big[\,\psi(n_i^{Z}+1) - \psi(n_i^{XZ}+1) - \psi(n_i^{YZ}+1)\,\Big], \tag{2}$$

where ψ is the digamma function, k is the neighbour count (parameter `k`, default 4), and N is the number of embedded points. The three counts are taken in the marginal sub-spaces of the conditioning set and its unions with each variable: n_i^{Z}, n_i^{XZ}, n_i^{YZ} count the points whose distance from point i in the Z, (X,Z), and (Y,Z) sub-spaces respectively is strictly less than ϵ_i. This is the .ksg_cmi function; the transfer entropy $T_{Y\to X} = \widehat{I}$ at $Z = X_t^{(\ell)}$ is .te.

The max-norm and the neighbour radius. The metric is the maximum (Chebyshev) norm,

$$\|u - v\|_\infty \;=\; \max_c |u_c - v_c|, \tag{3}$$

taken over all coordinates of the relevant space. The radius ϵ_i is the max-norm distance from point i to its k-th nearest neighbour in the *full* joint space (X,Y,Z), self excluded. The engine works in raw return coordinates and does not standardise, because the G20 series are already daily log-returns and the max-norm is not scale-invariant, so a z-score would change the estimate; the runners and the GPU helper both note this. Computing ϵ_i exactly under the max-norm on a tree built for the Euclidean norm uses the inclusion $\|\cdot\|_\infty \le \|\cdot\|_2 \le \sqrt{d}\,\|\cdot\|_\infty$: an inflated L_2 candidate list is pulled, exact Chebyshev distances are recomputed, the k-th smallest is taken, and a point's radius is accepted only once its candidate list provably covers that radius (.maxnorm_knn_dist). This makes the radius exact, not approximate.

The one-dimensional count boundary. A subtlety the bit-exactness gate caught, and which a careful reader should know about, lives in how the strict-inequality counts are realised. For a sub-space of dimension ≥ 2 the count is taken directly as $\#\{j \ne i : \|u_i - u_j\|_\infty < \epsilon_i\}$ by recomputing the max-norm and applying $<$ (.count_kd). For a one-dimensional sub-space (which arises for the Z count when $\ell = 1$) the engine uses a sorted-array realisation via findInterval (.count_1d), testing membership against the half-open interval $[v-\epsilon,\, v+\epsilon)$ rather than the absolute form $|v_j - v| < \epsilon$. The two disagree on exactly one floating-point boundary tie: when $|v_j - v|$ rounds to exactly ϵ but $v - \epsilon$ rounds down, the interval form *includes* the point that the absolute form rejects. The engine's choice is the .count_1d convention, and the GPU helper reproduces that convention host-side rather than the textbook absolute form (`gpu/ksg_gpu.py`, _count_1d_engine_batch), so the observed estimate matches the engine's own six-decimal emission exactly. The lesson, recorded as a learning, is to gate on the estimator's own output including its floating-point boundary conventions, not on the textbook formula.

Effective transfer entropy. The raw estimate of Eq. (2) carries a finite-sample bias that does not vanish under the null of no information flow. Following the surrogate bias-correction of Marschinski and Kantz [29], the *effective* transfer entropy subtracts the mean of the estimator over surrogate source series that share the source's marginal and linear autocorrelation but carry no directed dependence on the target:

$$T^{\mathrm{eff}}_{Y\to X} \;=\; T_{Y\to X} \;-\; \frac{1}{B}\sum_{b=1}^{B} T_{\tilde{Y}^{(b)}\to X}, \qquad \tilde{Y}^{(b)} \sim \mathrm{IAAFT}(Y), \tag{4}$$

with B surrogates drawn by the IAAFT of §3.1. The same surrogate ensemble gives the permutation p-value $p = (1 + \#\{T_{\tilde{Y}\to X} \ge T_{Y\to X}\})/(B+1)$. The published NAMH pipeline

computes Eq. (4) explicitly (TE_eff in `r/namh_pipeline.R`); the deterministic namh_te method returns the raw estimate only and states that the effective TE and the p-values, being surrogate-dependent and unseeded in the package, are not bit-reproducible and belong to the async pipeline.

GPU equivalence on the observed estimate. The cost is an $O(N^2)$ max-norm neighbour search, once per directed pair and once per surrogate, which previously saturated the workstation. The search now runs on the RTX 3000 Ada GPU through two numba.cuda kernels (one for the radii, one for the three counts), with the IAAFT surrogates batched on the host (`gpu/ksg_gpu.py`). Crucially, the *observed* transfer entropy is the only quantity the evaluation suite gates, and it is held bit-exact to the CPU estimator: the measured maximum discrepancy is 7×10^{-16} and the six-decimal emission is identical, across $\ell \in \{1, 2\}$ and $k \in \{4, 5\}$, verified by `gpu/equiv_gate.R`. The p-value is an unseeded surrogate draw in both the CPU and GPU paths and is never claimed reproducible. Defaulting to the GPU therefore can never move a published number; a box with no CUDA device, or a forced `CE_GPU=0`, degrades silently to the governed CPU path.

3.4 Determinism and reproducibility of the estimators

Reproducibility of a Monte-Carlo estimator requires that two facts be pinned: the random stream, and the order of reduction. The engine pins both.

The random stream. Where a method's result depends on a draw, the runner sets `RNGkind("L'Ecuyer-CMRG")` and `set.seed(42)` before the computation. The combined multiple-recursive generator of L'Ecuyer [27] is chosen because it splits into independent sub-streams that parallel::mclapply can hand to its fork workers deterministically: under a fixed seed each child stream derives reproducibly from the master seed. The canonical surrogate count is $B = 200$. The published NAMH end-to-end run is the clearest case (`r/namh_pipeline.R`): the namh package's own IAAFT generator is unseeded, so the paper's published surrogate run is not bit-reproducible; the engine's runner pins the seed and the generator so that its result *is* reproducible for a fixed $\{\text{seed}, n_{\text{cores}}\}$ pair. That dependence on the core count, through the stream split, is stated rather than hidden. The two ksg methods are deliberately *not* seeded, so their surrogate p-values are an honest non-reproducible draw in both the CPU and GPU paths; their reproducible content is the observed TE, which carries no randomness.

The order-free reduction. The directed-pair and grid loops are order-free: the result is a set of per-pair estimates reduced by operations (sums, means, rank correlations, set overlaps) that do not depend on the order in which the pairs complete. This is what lets the worker count be governed without touching a number. The shared helper ce_ncores in `r/_io.R` sets the mclapply budget from a base of `detectCores()` minus a reserve, overridden by an explicit `n_cores`, and clamped last by a hard ceiling `CE_MAX_CORES` (which the tower sets to $\lfloor(\text{cpus} - 2)/\text{MAX\_CONCURRENT}\rfloor = 10$ on the workstation). BLAS threading is pinned to one thread per spawn. Because the loops are order-free, changing the worker count from one to ten never changes a TE value; it only governs CPU. This is the formal content of an invariant the system depends on.

Invariant 2 (Worker-count neutrality)**.** For every method that reduces a set of independently computed per-pair or per-grid estimates, the emitted result is invariant to the number of parallel workers and to the order in which they complete. Core governance and BLAS pinning therefore change CPU occupancy, never a published value.

3.5 A failable test per method

The discipline that makes the registry trustworthy is one rule, applied without exception.

Principle 3 (Catalogue entry requires a failable test)**.** A method enters the registry only with a pre-registered, failable acceptance band on its own output, fixed before the run. A regression turns the public dashboard red rather than passing silently; a hole is marked, never filled; and the band is never edited to fit a result.

Three verified examples show the principle at work, drawn from Table 3 and the registry. They are stated with their provenance, in the registered measured voice, because a real number with a band is worth more than an adjective.

Example 1 (An observed estimate in band)**.** The ksg_te method on the 18-market G20 panel returns the USA→Japan transfer entropy as **0.154234**, across 306 directed pairs of which 108 are significant at $p < 0.05$ against IAAFT source surrogates. The pre-registered band is $T \in [0.150, 0.158]$; the value sits inside it. The same observed estimate is the quantity the GPU path reproduces to 7×10^{-16}, and a full-panel re-run of all 306 pairs reproduced 0.154234 exactly.

Example 2 (A stability claim across nuisance parameters)**.** The ksg_robustness sweep re-runs the same estimator across the grid $k \in \{3, 4, 6, 8\} \times \ell \in \{1, 2\}$ and checks two pre-registered conditions: that the headline directed edge is ranked first at every grid point, and that the mean Spearman rank correlation of the full directed-TE vector against the baseline lies in $[0.6, 0.85]$. Both hold: the mean Spearman is **0.7282** (minimum 0.581), and USA→Japan is the #1 directed edge across all eight (k, ℓ) settings. The headline link is therefore robust to the estimator's nuisance parameters, not an artefact of one choice.

Example 3 (A published estimate held to a tight band)**.** The namh_hurst method, running the published namh package on its rolling-window panel, returns a mean Hurst exponent for Gold of **0.490** (0.4899). The pre-registered band is $[0.4889, 0.4909]$, a tight window that would fail loudly on any drift in the package or the data. The value is reproduced to $\max|\Delta|\ 1.55 \times 10^{-15}$ same-machine by the namh_reproduce Δ-verifier.

Honest negatives are themselves results under Principle 3 and are immutable facts of the record, not failures to be hidden. The MCPFM out-of-sample call returns AUC 0.581 (DeLong $p = 0.030$) in version 1; a pre-registered version-2 re-evaluation returns AUC 0.470 (Hansen SPA $p = 0.031$), and that lower figure is reported openly with a reproduce status of honest-pending. The NAMH pipeline's own Benjamini–Hochberg FDR gate retains 0/552 directed edges at the canonical configuration, so its masked network, fixed point, and communities are degenerate by construction and are emitted as measured, in the dashboard's honest amber, rather than magnitude-dressed. A method that asserted a clean network there would be the failure; reporting the empty gate is the discipline.

4 Heterogeneous compute: core governance, GPU offload, and the asynchronous tower

The methods in the substrate are not uniform in cost. Most return in well under a second on a stored panel; a few – the transfer-entropy estimators in particular – spend their time in an $O(n^2)$ nearest-neighbour search and, run carelessly, can saturate a workstation. This section describes how Econstellar runs that uneven load safely. There are three concerns and one rule that ties them together. The rule is that *the number of workers must never change a number*: every heavy loop in the engine is an order-free reduction over independent directed pairs or grid cells, so how we split the work governs only the use of the machine, never the result. Given that rule, the three concerns are how many CPU cores a job may use (core governance), whether the

heaviest primitive can be moved off the CPU entirely (GPU offload), and how a job that runs for minutes rather than milliseconds is managed without blocking the synchronous kernel (the asynchronous tower).

4.1 Core governance: one budget, wired everywhere

The motivating failure is concrete and was diagnosed to a single line. Three methods – ksg_te, ksg_robustness, and sri_daily – each hard-coded their worker count as `ncores <- detectCores() - 1L`. On the 22-logical-core workstation that is 21 `mclapply` workers per job, and it ignored the job's own `n_cores` request entirely. With the tower configured to run two jobs at once and the numeric libraries left multi-threaded, two concurrent ksg_robustness jobs forked $2 \times 21 = 42$ worker processes, each of which could in turn spawn its own BLAS threads on top of the fork. That multiplicative oversubscription pinned all 22 cores at 100% and hung the machine; the reboot left two ksg_robustness records stranded in the `running` state. The fault was not the estimator but the absence of a single place that decides how many cores a job may use.

The fix is exactly that single place. In `r/_io.R` we added one governed budget, ce_ncores(p, *reserve*), and wired it into all six `mclapply` call sites. Its precedence is fixed and the ceiling is clamped last:

```r
ce_ncores <- function(p = NULL, reserve = 2L) {
  b <- max(1L, parallel::detectCores() - as.integer(reserve)) # 1. base
  if (!is.null(p) && !is.null(p$n_cores)) { # 2. job override
    req <- suppressWarnings(as.integer(p$n_cores))
    if (!is.na(req) && req >= 1L) b <- req
  }
  cap <- suppressWarnings(as.integer(Sys.getenv("CE_MAX_CORES", "")))
  if (!is.na(cap) && cap >= 1L) b <- min(b, cap) # 3. HARD ceiling, last
  max(1L, b)
}
```

Listing 2: The single core budget (`r/_io.R`).

A base of `detectCores() - reserve` can be raised or lowered by an explicit `n_cores` parameter, but the environment ceiling `CE_MAX_CORES` is applied afterwards and cannot be exceeded by any caller. The function always returns at least one. The complementary half lives in the tower process manager. The job server in `server/job-server.mjs` sets, for every R subprocess it spawns,

$$\texttt{CE\_MAX\_CORES} \;=\; \left\lfloor \frac{\texttt{cpus} - 2}{\texttt{MAX\_CONCURRENT}} \right\rfloor,$$

which evaluates to **10** on the 22-core workstation (reserving two threads for the operating system and the Node server). Because the ceiling is divided by the concurrency level, `MAX_CONCURRENT` jobs cannot jointly oversubscribe the box: two concurrent jobs now use at most 20 cores, never 42, and a job that asks for 21 under a ceiling of 10 is forced down to 10. The same spawn pins each numeric library to a single thread (`OPENBLAS_NUM_THREADS`, `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `VECLIB_MAXIMUM_THREADS`, `NUMEXPR_NUM_THREADS` all set to `1`), removing the second factor of the oversubscription. The resulting budget is surfaced in the tower's `/health` response and startup log as `core_budget`, so the governing number is observable rather than implicit.

Pinning the BLAS threads is not only a safety measure. A multi-threaded BLAS reduces in a non-deterministic order, so summation results can differ in their last bits from run to run; pinning each library to one thread fixes that order and makes the numeric result reproducible. The core-governance change therefore buys a reproducibility gain on top of the safety one, and it does so without any risk to the published numbers, because of the order-free structure of the loops it governs.

Invariant 3 (Worker count does not change a result)**.** Every loop governed by ce_ncores is a reduction over independent units – directed pairs for the transfer-entropy methods, (k, lag)

cells for the robustness grid – whose combination (a set of per-pair records, a mean, a ranking) does not depend on the order in which the units are evaluated or on how they are split across workers. Capping the worker count, or pinning BLAS to one thread, therefore changes only CPU occupancy and floating-point reduction order, never the estimate the eval suite gates.

4.2 GPU offload of the transfer-entropy search

Core governance makes the CPU path safe; the GPU path removes the heaviest load from the CPU altogether. The hot primitive is the Kraskov–Stögbauer–Grassberger estimator in its Frenzel–Pompe conditional-mutual-information form [21, 25, 33]: for each point we find ϵ_i, the max-norm distance to its k-th neighbour in the joint $(\mathrm{dst}_{t+1}, \mathrm{src}_t, \mathrm{dst}_t)$ space, then count neighbours with marginal max-norm distance strictly less than ϵ_i in three sub-spaces. That search is $O(n^2)$ and is exactly the work that hung the workstation. We ported it to two `numba.cuda` kernels (CUDA 12.4, numba 0.63.1 [26]) and run it on an NVIDIA RTX 3000 Ada Laptop GPU (compute capability 8.9, 8 GB). One thread handles one (search, point) pair, so a directed pair's observed estimate and all of its surrogates run in a single launch, amortising the launch and host–device transfer overhead. The kernels compute in FP64.

The choice of FP64 over FP32 is deliberate and is the crux of why a faster device does not threaten the engine's discipline. The Ada GPU's FP32 path is faster still, but the KSG estimate is the anchor that the eval suite checks; an approximate value would move that anchor and break the gate. We therefore pay for double precision so that the GPU result is not close to the CPU result but *identical* to it. Max-norm distances are computed by absolute value, subtraction, and maximum, all exact IEEE operations, so the k-th order statistic of an identical multiset and the strict-$<$ integer counts are the same on host and device.

That identity is enforced by a failable gate, `gpu/equiv_gate.R`, which runs the GPU helper and the engine's own CPU estimator on a small bounded case and exits non-zero unless they agree. The observed maximum absolute difference is at most 7×10^{-16} – it was 6.7×10^{-16} in the NAMH re-estimation – and the six-decimal emitted values are bit-for-bit identical. The gate caught a subtle and instructive discrepancy. The engine's one-dimensional neighbour count, used for the conditioning sub-space at lag = 1, tests membership with `findInterval(v - eps)` rather than the absolute form $|dp - v| < \epsilon$. On the rare boundary tie where $|dp - v|$ rounds to exactly ϵ but $v - \epsilon$ rounds down, the interval form *includes* a point that the absolute form excludes. The GPU helper replicates the engine's interval form host-side for that one-dimensional case, so the published estimate (the eval anchor) is preserved exactly rather than merely approximately. The lesson is that when a published estimator is ported to a faster device, the gate must be the estimator's own output, including its floating-point boundary quirks, not the textbook formula.

Principle 4 (Gate on the estimator's output, not its textbook form)**.** The only quantity the eval suite gates from the transfer-entropy methods is the observed TE (the `ksg_te` band and the `ksg_robustness` rankings and Spearman correlations). Because the GPU reproduces that quantity bit-for-bit, GPU execution can be the default without ever moving a published number. The significance p-value is a different matter: it comes from IAAFT surrogates [34], and the engine seeds neither transfer-entropy method, so the p-value is a non-reproducible draw on the CPU path and on the GPU path alike. We never claim it reproducible, and the gate never tests it.

Dispatch and fallback are handled by `ce_ksg_gpu_pairs` in `r/_ksg_core.R`, shared by `ksg_te` and `ksg_robustness`. The default is `AUTO`: setting `CE_GPU=0` forces the CPU path, otherwise the dispatcher probes for a usable CUDA device, and if one is present it writes the returns matrix as a raw column-major binary alongside a JSON spec (n, k, lag, surrogate count, the pair list), invokes `gpu/ksg_gpu.py`, and parses the per-pair results, stamping a `compute_path` field. Robustness is the point of the design: any failure at all – `CE_GPU=0`, no CUDA device, a missing or broken helper, a malformed or short result – returns `NULL`, and the caller then runs its governed

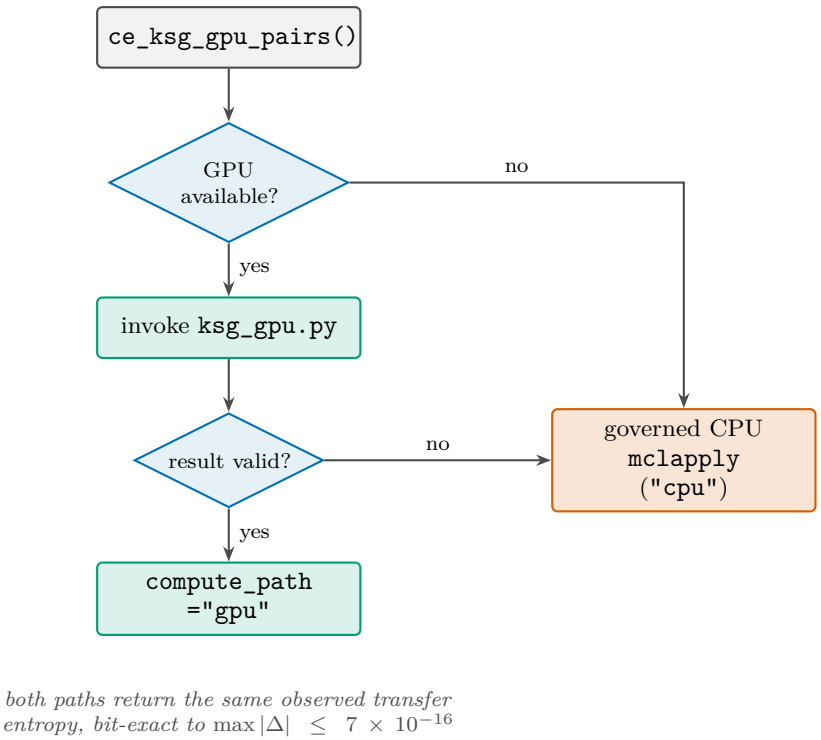

Figure 4: GPU dispatch with governed-CPU fallback. "GPU available?" tests `CE_GPU`≠0, a present CUDA device, and a successful probe; any failure, including a malformed result, falls back to the path that core governance already made safe. The offload changes only where the observed transfer entropy is computed, never its value.

CPU `mclapply` path. A box with no GPU, or one with a broken toolkit, degrades silently and safely to the path that core governance already made safe.

The surrogate inner loop stays on the host. IAAFT is dominated by a per-surrogate FFT rather than by the neighbour search, so the helper runs it batched over surrogates with a batched FFT and feeds the resulting source series into the same two kernels. The measured effect of the offload is twofold – it is faster and it frees the CPU. On this workstation, 12 directed pairs with $B = 99$ surrogates run in 82 seconds at roughly one core on the GPU against 289 seconds across roughly seven cores on the governed CPU, a **3.5**× speed-up that also returns six cores to other work. In the isolated trial on a deterministic coupled system the kernel search alone runs up to **73.76**× faster ($n = 3000$), and between 32× and 74× faster across $n \in \{2000, 3000, 5000\}$, all bit-exact. The confirmation that matters is the full-panel re-run: the exact workload that hung the machine – the full G20 panel, 306 directed pairs at $B = 99$ – completed on the GPU in **35.6 minutes** at 100% of *one* core with a peak of 388 MB of host memory, returning 106 of 306 pairs significant and the USA→Japan estimate at **0.154234**, identical to the CPU value. The saturation that hung the 22-core box is gone, and the heaviest, most parallel primitive in the engine now runs on the device best suited to it.

4.3 The asynchronous tower

A run that takes thirty-five minutes does not belong on the synchronous kernel, which runs scale-to-zero on Cloud Run under a request timeout. Four methods are heavy enough to need a different home: ksg_te, ksg_robustness, namh_pipeline, and soch_robustness, each flagged `long_running` in the registry. These run on the always-on workstation tower managed by `server/job-server.mjs`, which executes the same parameterised-only R registry as the kernel – it discovers methods from `r/*.R` and refuses anything else – but asynchronously. A `POST /api/jobs/submit` validates

the workspace and the method name, returns a `job_id` immediately, and queues the work; the manager runs at most `MAX_CONCURRENT` jobs at a time under the core budget of Section 4.1. Each job record and its result persist to a `.jobs/` directory on disk and are reloaded on startup, so a job survives a restart and is addressable by a permalink for thirty days; a job left `running` by a crash is requeued rather than lost.

Progress is reported without compromising the engine's output contract. The kernel requires that a method's standard output be a single pure JSON object, so progress lines cannot simply be printed. The progress emitter ce_progress in `r/_io.R` is therefore silent unless the environment variable `CE_PROGRESS` is set, and only the tower sets it:

```
ce_progress <- function(fraction = NA, stage = "", elapsed_s = NA) {
  if (!nzchar(Sys.getenv("CE_PROGRESS"))) return(invisible(NULL)) # silent under the kernel
  cat(toJSON(list(`__progress__` = TRUE, fraction = fraction,
                  stage = stage, elapsed_s = elapsed_s),
             auto_unbox = TRUE, na = "null"), "\n", sep = "")
  flush(stdout())
}
```

Listing 3: Progress is opt-in, so the synchronous contract is never polluted (`r/_io.R`).

Under the synchronous kernel this is a no-op and the stdout remains a single JSON object. Under the tower, which spawns each method with `CE_PROGRESS=1`, the method emits newline-terminated progress objects tagged `__progress__`; the job manager parses those lines, updates the job record, and pushes them to any subscriber over Server-Sent Events at `/api/jobs/:id/stream`, while the final unterminated JSON object is read as the result. The same heavy method thus serves two callers – a synchronous one that sees only the answer and an asynchronous one that sees the answer and the progress – from one code path, with the environment, not the source, deciding which behaviour is in force.

5 The AI layer: a natural-language client, not a privileged path

The engine is reachable in plain language. A researcher can ask a question – "is the UK equity series stationary?", "which market sends the most information to Japan?" – and receive an answer that ran the real econometrics rather than a plausible paragraph. Two routes provide this. `/api/chat` is a fast analyst backed by Gemini 2.5 Flash; `/api/research` is a deeper assistant backed by Gemini 2.5 Pro that both runs analyses and grounds its synthesis in literature. The design problem they pose is not capability but containment: a language model that can trigger compute is a new way to reach the engine, and we must be sure it is not a new way to reach anything else. The whole of this section is organised around the claim that it is not, because the AI layer is a client of the engine on exactly the same footing as a script or a web form, with no execution surface of its own.

5.1 The chat route

`/api/chat` is a single function-calling turn followed by a summary. The user's message goes to Gemini with one tool, run_analysis, whose parameters are a method name and its typed arguments; the model either answers conceptually with no tool call or emits exactly one call. That call is not executed as written. It is passed through the same validate routine every other caller uses – which rejects an unknown method or a mistyped parameter – and only the validated, cleaned method and arguments reach runSandboxed. The result is then handed back to the model for a second turn that explains the numbers in prose. The route returns `{reply, ran}`, where `ran` records the method, the cleaned parameters, the raw result, and the wall time, so the natural-language answer is always accompanied by the machine record it was built from. The

system prompt is a careful econometrician, not a generic assistant; in particular it carries the stationarity rule discussed below, so the model never reports a price level as stationary.

5.2 The research route

`/api/research` is the same idea at publication depth, and it runs in two phases. Phase A is an agentic loop, bounded to a few steps, in which Gemini 2.5 Pro is given the run_analysis tool and runs live econometrics wherever the question admits it – it may issue several calls, see their results, and call again, each call passing through the same validate and runSandboxed path as the chat route. Phase B is a grounded synthesis: the analyses executed in Phase A are summarised into a digest, the digest and the question are sent back to the model with a `google_search` tool, and the model writes the final answer grounded in real sources. The route returns

`{answer, citations[], searches[], analyses[], steps}`,

so the caller receives the prose answer, the grounding citations (title and URI), the search queries the model issued, the full record of every analysis it ran, and the step count. The citations are read from the model's grounding metadata, not invented by the prose.

The two phases are kept separate for a concrete reason of API behaviour rather than preference. Function-calling and `google_search` grounding are not reliably combined within one Gemini request, so the engine does not attempt to. Instead it runs the tools in their own passes and carries the executed analyses forward as text into the grounded pass; the model still cites real literature *and* reasons over real live numbers, but the two tool modes never share a single request. This is the kind of fragility a systems report should name: the separation is a workaround for a model-API constraint, documented in the route itself.

The system prompt for the research route does substantial work, and all of it is fact, not flourish. It injects the eight-primitive substrate – long-memory estimation, MODWT wavelet decomposition, transfer entropy, surrogate inference, community detection, network-formation game theory, structural attribution, and classifier validation – so the model reasons in the engine's own vocabulary; it names the frameworks those primitives instantiate; and it injects the verified-results table verbatim with the instruction to quote it exactly and never alter it, so a number the model states is a number the engine has checked. It also carries the stationarity rule as an inviolable constraint.

Principle 5 (The stationarity rule)**.** Transfer entropy and the other dependence measures are estimated on $I(0)$ log-returns, never on $I(1)$ price levels. Equity price levels carry a unit root (a random walk with drift); log-returns, $\mathrm{diff}(\log \mathrm{price})$, are typically stationary. The stored G20 panel is already log-returns, so any "stationary" verdict over it concerns returns, and the model is required to say so explicitly ("UK equity *returns* are stationary", not "the UK market is stationary"). Embedding this rule in the prompt prevents the most common and most consequential error an unguided model would make on financial series.

5.3 The safety argument

The reason the AI layer can be exposed at all is that it adds no power. Everything it can cause to happen, a direct caller of `/api/compute/run` could already cause to happen; everything a direct caller cannot do, the AI layer cannot do either. This is worth stating precisely, because it is the load-bearing claim of the whole design.

Invariant 4 (The AI layer is a client, not a privileged path)**.** The model cannot execute arbitrary code. It can only name a method already in the registry and supply typed parameters for it, and it can only read that method's returned result. Its proposed call is validated by the same validate routine and executed by the same runSandboxed path as any other request, so it inherits every

guard those impose and introduces no new execution surface. The registry of parameterised methods is the boundary; the language model sits outside it, on the same side as a script or a browser.

This follows directly from the implementation. A function call from Gemini is data: a method string and an arguments object. It is subjected to validate before it is allowed to mean anything, and validate is the same gate the public compute route uses, so an attempt to name a method that does not exist, or to pass a parameter of the wrong type or shape, is rejected before any R process is spawned. There is no path by which the model can supply code, a file path, a shell fragment, or a network target; the parameterised-only registry that is the engine's primary guard against arbitrary execution applies to the model exactly as it applies to everyone else. The AI layer widens the engine's audience – it makes the methods reachable by people who would not write the JSON by hand – without widening its attack surface.

5.4 Rate limits and the cost bound

Because each model turn costs real money, the AI routes are bounded on several axes, and the bound that matters most is a hard ceiling on spend rather than a courtesy throttle. The per-IP limiter caps `/api/research` at roughly five requests per minute and roughly twenty per day, with `/api/chat` held to a higher but still bounded rate; these stop a single client from monopolising the service or slowly draining the credit. Above those sits the ceiling that bounds the worst case regardless of how many clients attack at once. Each instance permits at most 400 paid LLM turns per UTC day, and Cloud Run is configured for at most two instances, so the fleet-wide maximum is

$$400 \text{ turns/instance/day} \times 2 \text{ instances} \;=\; \textbf{at most 800 paid turns/day}.$$

The day counter resets at UTC midnight, and when the cap is reached the routes return a service-unavailable response with a retry hint rather than paying for another turn. The limiter is process-local and resets on restart, which is acceptable precisely because the per-instance daily cap multiplied by the instance ceiling already gives a bounded fleet-wide daily maximum: the AI layer cannot, under any traffic pattern, spend more than that.

6 Verification and Epistemics

The engine's claim to trustworthiness does not rest on the authors' assurance. It rests on a small set of machine-run artefacts that any reader can inspect and, given the repository, re-execute: a pre-registered evaluation suite that grades live computation against failable numerical bands, a nightly loop that re-runs that suite and writes its own findings into a ledger it does not get to edit by hand, a claims record that demotes rather than deletes, and a thin formal layer in which one identity is closed in a proof assistant with an honest count of what remains open. This section describes those four artefacts and the single property that ties them together: the apparatus is built so that its own correctness, and the wiring that enforces it, are themselves things that can fail a test in public.

6.1 The evaluation suite

The suite lives in `evals/run-evals.mjs` and emits a single artefact, `evals.json`. Each catalogue method has exactly one row. A row names the method, the parameter object it is called with, an `expected` string stating the pre-registered band in plain language, a `source` citing where that band comes from, and a `check` function that extracts one quantity from the live result and returns a boolean. The runner posts the request to the deployed engine (or, for the four heavy asynchronous methods, submits a real job to the workstation job server and polls it to

completion), applies the check, and records the verdict. It computes; it does not read a cached number. The most recent full run recorded **26 of 26 rows passing** (0 fail, 0 pending) against the live service.

Three rules govern the artefact, and each is enforced by construction rather than by convention.

Principle 6 (Bands are pre-registered and failable)**.** A band is fixed before the run, from a documented verified value, and is written so that the result can fall outside it. Where a method's verified tuple does not pin a full configuration, the band degrades to an invariant check that can still fail on the wrong shape, a non-finite value, an unstable system, or a wrong verdict, never to a check that always passes.

Principle 7 (The artefact is never hand-edited)**.** `evals.json` is the output of one genuine execution of the committed runner. A red row is information, not a defect to be quietly corrected in the file. To change a number one re-runs the suite, not the artefact.

Principle 8 (A band failure halts; only extraction may be calibrated)**.** Failures split in two. A *band* failure means the engine is wrong: work stops and the cause is investigated. An *extraction* failure means the harness misread the result's shape: the check is corrected and the whole suite is re-run. Fixing extraction is calibration, not gaming, only when the expected band is left untouched.

The bands are not decorative, and several of them encode a number whose value is the point. Three rows make the discipline concrete. The first is an ordinary positive result. The synchronous transfer-entropy row ksg_te calls the estimator on the USA and Japan series and checks the directed edge:

```
{ m: "ksg_te", p: { series: ["USA", "Japan"] }, async: true,
  expected: "USA->Japan full-sample KSG TE in [0.150,0.158] (verified 0.1542)",
  src: "A2 + #39 anchor",
  check: r => { const e = (r.edges || r.pairs || [])
      .find(x => /usa/i.test(x.from) && /japan/i.test(x.to)) || {};
    const v = num(e.te ?? e.TE);
    return { value: v, pass: inBand(v, 0.150, 0.158) }; } }
```

The last full run returned 0.154234 for this edge, inside the band, from a real job on the tower (`job_20260612_92fa3f31`); the worked trace in Section 9 follows that exact call from HTTP request to single-JSON response. The second is a degenerate negative that must *stay* degenerate. The namh_pipeline row runs the full NAMH surrogate pipeline end to end under a fixed seed and checks that, under the paper's own Benjamini–Hochberg gate [4], the network is empty:

```
{ m: "namh_pipeline",
  p: { n_surrogates: 200, window_index: 1, seed: 42, n_cores: 2 }, async: true,
  expected: "FDR retains 0/552 under the paper's own gate (degenerate, honest amber);
             raw te_mean in [-0.2242,-0.2222]",
  check: r => { const w = (r.per_window || [])[0] || {}; const f = w.fdr || {};
    const c = r.config || {};
    return { value: num(w.te_raw && w.te_raw.mean),
      pass: num(f.n_edges) === 0 && num(f.n_possible) === 552
            && w.degenerate === true
            && inBand(num(w.te_raw && w.te_raw.mean), -0.2242, -0.2222)
            && c.seed === 42 && c.rng === "L'Ecuyer-CMRG" }; } }
```

Here a *populated* network would be the failure and the empty one is the pass: the row insists on **0 of 552** directed edges retained in the window, the window flagged `degenerate`, the raw window mean inside $[-0.2242, -0.2222]$ (the last run returned -0.22322165), and the generator pinned to L'Ecuyer's combined multiple-recursive scheme [27] at seed 42. The third is the live systemic-risk index. The sri_daily row recomputes the static-panel index in full from the stored

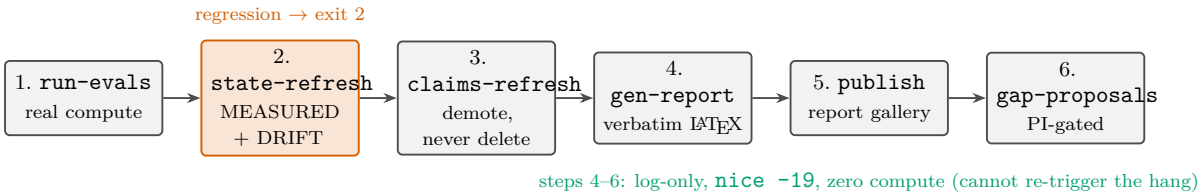

Figure 5: The six-step nightly loop. Step 2 governs the exit code, turning a regression into a non-zero exit and a red dashboard; steps 4–6 do no computation, so the reporting layer can never re-trigger a saturation. No step deletes a claim or edits a result.

panel and checks it against a band one part in 10^5 wide, $[0.00848, 0.00850]$, around the first honest point 0.00849 over 306 directed pairs. A reproduction target this tight can only be met by re-running the same computation, which is the property the row exists to certify.

The suite is honest about its own operational hazards, which is itself part of the discipline. The runner rides out a Cloud Run cold start with a second, longer health probe, because a single short probe once misreported a live engine as unreachable while all rows passed; it waits politely under the engine's rate limit rather than recording a self-inflicted red; and asynchronous rows are either real tower jobs with persistent identifiers or honest `async_pending` placeholders, never fabricated values. The artefact records, per row, the measured value, the elapsed milliseconds, whether the engine served it from cache, and any job identifier, so that the run can be audited line by line.

6.2 The nightly self-learning loop

The suite would be a static check were it not re-run. The centrepiece of the verification layer is the loop in `scripts/nightly-loop.sh`, installed under `cron` at `10 7 * * *` UTC, fifty minutes after the 06:00 UTC systemic- risk tick. Its design intent is to survive a reboot, to be loud in the cron log when something regresses, and never to silently repair a problem it should surface. The cron entry and the script are installed and self-tested; at the time of writing the loop has not yet completed an unattended run, so its cadence is reported as configured rather than demonstrated. It re-arms the long-lived job server only if its port is closed, killing nothing, then runs the six steps summarised in Table 4 and pictured in Figure 5.

Two properties of the loop are worth drawing out. The first is the division of authority between its steps. The eval suite and state-refresh are the canary: state-refresh exits 2 on any regression – the engine unreachable, a failed eval, a changed method count, or a systemic-risk feed stale beyond three weekdays – and that exit code becomes the loop's, so a problem is never silent. Everything downstream is deliberately log-only and non-fatal. A BigQuery outage in the claims step, a LaTeX fault in the report step, or a missing checkout in the publish step is recorded and skipped; none is allowed to repaint a night whose real signal is the suite and the state ledger above it. The report step is the one that can always yield, which is why it runs `nice -n 19`: it is pure rendering plus a single `pdflatex` pass, and it must never contend with a long-running tower job for the processor.

The second property is the one we hold to most firmly: the loop's own wiring is a failable eval. A separate test, `evals/loop-integrity.test.mjs`, asserts that the steps exist, that the artefact paths the steps pass to one another agree, that the set of claims the refresher maps is a subset of the suite's rows, that no step on the nightly path issues a delete, that the gap rule and its PI gate are intact, and that the installed state is reported honestly. We record this as an invariant because it is the load-bearing one.

Invariant 5 (The verifier verifies itself)**.** The self-checking loop is subject to the same standard as the methods it checks: its wiring – step existence, artefact-path agreement, the claims-to-

Step	Script	What it does, and how it stays honest
1	`run-evals.mjs`	Re-arms the job server (port-guarded, no kills), then runs the full suite, async rows as real tower jobs, writing a fresh `evals.json`.
2	`state-refresh.mjs`	Measures live health, the eval summary, and the SRI feed; rewrites *only* the `MEASURED` block of `STATE.md` and appends a dated `DRIFT` line on any regression. Its exit code is the loop's: non-zero means a problem line was written.
3	`claims-refresh.mjs`	Re-verifies the mapped claims against the same night's artefact: a pass bumps `last_verified`; a red marks the claim `contested` and inserts the other side of the pair. Never deletes. Log-only, so a database outage cannot repaint the night.
4	`gen-compute-report.mjs`	Renders a LaTeX run report verbatim from `evals.json`; never recomputes, never invents, renders failures as loudly as passes. Run `nice -n 19` so it always yields to a tower job.
5	`publish-reports.mjs`	Copies each report PDF and a thumbnail into the static report gallery and rewrites its manifest. Pure file copy, `nice`'d, non-fatal.
6	`gap-proposals.mjs`	Mines the demand signals the engine exposes, applies the pre-registered rule, and emits PI-review candidate skeletons. Never auto-deploys.

Table 4: The six steps of `scripts/nightly-loop.sh`. Step 2 owns the loop's exit code, so a regression is loud in the cron log; steps 4–6 are log-only and non-fatal, so a rendering or reporting fault cannot mask the canary in steps 1–3.

evals subset relation, the no-delete rule, and the pre-registered, PI-gated gap rule – is itself a pre-registered, failable test. The apparatus that asserts correctness can therefore fail loudly when its own assumptions break.

That this is not a formality is visible in the test's history: its first execution was red, eleven of twenty, and all of the failures were the harness misreading shapes (a `m:` key written `method:`, a quoted path, a `DELETE` appearing inside a comment). Those were extraction failures, calibrated per the band-versus-harness rule with the bands untouched, and the test then went green – exactly the discipline the suite imposes on every other row, now imposed on the loop itself.

6.3 The learning ledger

The engine keeps a single state file, `STATE.md`, with three blocks. The `MEASURED` block is machine-written by state-refresh and is never touched by hand; the `LEDGER` block is appended by working sessions; the `LEARNING` block stages each burned lesson through a fixed lifecycle – *fail* (the lesson enters when something breaks), *investigate* (the cause is named), *verify* (the fix is proven), and *distill* (the lesson is promoted to a consultable Skill). Thirteen learnings are recorded; Table 5 states each in a line.

Three of these are load-bearing for the systems story told elsewhere in this paper. L11 is the resolution of the core-governance hang: two concurrent robustness jobs each forked one worker per logical core, 2×21 workers against unpinned multi-threaded linear algebra, and the fix was a single governed budget – one server-set core ceiling clamped last, with the BLAS thread count pinned to one, applied at every parallel fan-out site. The pin is a determinism gain as much as a stability one. L12 and L13 are the GPU bit-exactness bar: the nearest-neighbour search that hung the workstation ports to two CUDA kernels, and the gate insists the device *equal* the CPU estimator rather than approximate it. The non-obvious lesson distilled there is that one must gate on the estimator's own output, including its floating-point boundary behaviour, not on the textbook form of the estimator – a one-dimensional neighbour count using `findInterval` includes a boundary point that the absolute-difference form excludes, and the gate caught it.

ID	Date	Stage	Lesson (terse)
L1	2026-06-11	distilled	A robustness grid must anchor through the paper's own scripts and estimation universe before any badge is written.
L2	2026-06-11	distilled	Moving the engine root silently changes API-key resolution and can ship a chat-disabled revision; check the WARN line.
L3	2026-06-10	distilled	Yahoo serves `close: null` for hours after a session; the finite-close guard makes the tick skip honestly. The schedule is the guard.
L4	2026-06-11	distilled	Eval failures split into band failures (stop, investigate) and extraction failures (fix the check); calibration only with the band untouched.
L5	2026-06-11	distilled	A documented tuple must name its full parameterisation or it is not a reproducible claim (the five-market panel).
L6	2026-06-08	distilled	LaTeX source grep is insufficient for fact consistency; verify against the rendered PDF, including figure labels.
L7	2026-06-12	distilled	A reboot killed both daemons mid-pipeline; long-lived services need a reboot-surviving re-arm, and pipelines should write artefacts as they go.
L8	2026-06-04	distilled	Bibliography titles are looked up, never reconstructed from a codename; verify id, title, and authors against the registered record.
L9	2026-06-12	verify	A dual-use script must guard its CLI dispatch with the main-module check, or an import can fire the wrong routine.
L10	2026-06-12	verify	Grep a large vendored dependency's own source for exact symbol names before writing against it (the Lean proof compiled first try).
L11	2026-06-13	verify	The 22-core hang had one cause: hard-coded `detectCores()-1` fan-out under concurrency; fixed by one server-set core ceiling with pinned BLAS.
L12	2026-06-13	verify	GPU offload is feasible now with no install; the hung k-NN primitive ports to two CUDA kernels and is bit-exact to the FP64 CPU reference.
L13	2026-06-13	verify	`ksg_te` and `ksg_robustness` now dispatch to the GPU, gated on the estimator's own output including its floating-point boundary quirk.

Table 5: The learning ledger `STATE.md` L1–L13. A lesson enters at *fail*, is named at *investigate*, proven at *verify*, and promoted to a Skill at *distill*.

Together these three turned an unbounded CPU fork into a governed budget plus a bit-exact device offload, and each remains in the ledger at *verify* pending the gated promotion to a Skill.

6.4 The claims ledger

Above the per-run artefact sits a slower record: what the engine takes itself to know. The claims ledger is a BigQuery table whose schema is fixed in `epistemic/schema.json`. Each row is one claim, with a stable `claim_id`; a `type` drawn from {empirical, methodological, formal, robustness, hole}; a `statement` written in one paragraph including its own limits; an `established_at` and a nullable `last_verified` timestamp; a nullable `confidence` that carries a badge pass-rate where the claim is badge-derived and is `NULL` otherwise, never invented; repeated `conditions` and `counter_conditions`; repeated `provenance_ids` so that every claim traces to a job, a badge, a file anchor, or a build; repeated `paper_refs`; and a `status` of *established*, *contested*, or *superseded*.

The ledger is seeded from verified ground truth alone by `scripts/claims-seed.mjs`, which carries roughly twenty provenanced claims and validates each against the schema before any write. The seeded set is deliberately catholic about what counts as knowledge: it includes empirical

results (the USA-to-Japan transfer entropy; the Table-5-exact channel shares), a formal claim (the closed spectral identity below), methodological claims (that the live index shares only a name, not a validation, with the published systemic-risk index), and *holes* stated as claims about the programme's own gaps (that the WaveQTE Stage-1 estimator is structurally unavailable in the image and the engine's variant is a labelled reimplementation). Honest negatives and marked holes are first-class rows, not omissions.

The lifecycle is the epistemic core. The nightly refresher `scripts/claims-refresh.mjs` re-verifies the mapped claims against the same night's artefact and obeys one rule above all others:

Invariant 6 (The record demotes, it never deletes)**.** A passing eval bumps an established claim's `last_verified`; a failing eval marks the parent claim `contested` and inserts an auto-generated contradiction as the other side of the pair, linked by a `contests:` provenance id. Nothing on the nightly path is ever deleted. A contested row is not silently re-greened by a later pass: only PI review resolves it, and the revision history – including a claim that was once contested and is now superseded – is part of the knowledge, not noise to be cleaned away.

The WaveQTE divergence is the worked instance of the full lifecycle. A 2026-02 working paper reports the financial channel dominant in the global financial crisis at 50.0%, while the engine's verified ground truth, the published package reproduced to 0.000 pp, gives the trade channel dominant at 27.9%. The claim was seeded *contested*, with both sides recorded, then *superseded* once the investigation established that the two figures are different estimands from different methods on different partitions, not a contradiction. The superseded row ships; it is never deleted, and a separate established claim records the resolution and links back to it. The claims are exposed read-only over the kernel at `GET /api/claims` (and `/api/claims/:id`), so the contested and superseded rows are publicly visible and the record fabricates nothing when its backing store is briefly unavailable.

6.5 The formal layer

The thinnest and strictest layer is a formal one. The SOCH element rests on a spectral identity: the squared modulus of the cascade transfer function equals a product-Lorentzian spectrum. We close that identity in Lean 4 against mathlib, as the proposition `prop:spectrum`,

$$\|H(\omega)\|^2 = S(\omega),$$

machine-checked unconditionally. The continuous-integration check is a green `lake build`; the reproduce page greps the Lean source and fails on drift, so the public count and the source stay in lockstep. The stationary-point half of the companion peak lemma is also fully accepted: the first-order condition is shown equivalent to a cubic in ω^2, the symmetric peak is located at $\omega^* = \alpha/\sqrt{3}$, and exactly one positive stationary frequency is shown to exist and to lie in a stated bracket.

What we will not do is dress the layer as more complete than it is. The analytic half of the peak lemma – that the maximiser of $\omega\, S(\omega)$ exists and satisfies the first-order condition – remains stated, not proved.

Invariant 7 (A `sorry` is never reported as a proof)**.** The formal layer carries an honest `sorry`-count of **1 of 12** declarations. The single open obligation is named and its route is documented in the source; it is never presented as closed, and the public count is held to the source by the same lockstep check that gates every other figure.

This is the same posture as the rest of the section, applied to machine-checked mathematics rather than to a numerical band. A closed identity is reported as closed because a proof assistant accepted it; an open obligation is reported as open because none did. The four artefacts – the

failable suite, the self-checking loop, the demoting claims record, and the honestly counted proof – are one mechanism seen at four time-scales, and the property they share is the one the project exists to demonstrate: every figure the engine states is a figure the engine can be seen to fail.

7 The Elements

The engine is the vehicle; the elements are what it carries. Six published frameworks have come out of the group, and it would be possible to read each as a self-contained study with its own codebase. We read them instead as elements of one programme: distinct economic questions, answered by distinct configurations of the same eight-primitive substrate, certified by the same failable gate, and re-runnable through the same engine. The substrate primitives are long-memory estimation, the maximal-overlap discrete wavelet transform (MODWT), transfer entropy, surrogate generation by the iterated amplitude-adjusted Fourier transform (IAAFT), community detection, network formation, structural channel attribution, and receiver-operating-characteristic (ROC) evaluation. Each element below states the economic question it asks, the primitives it instantiates, its headline verified result, and its honest status. The numbers are quoted exactly as the verified record holds them; where a figure is not in that record we mark it rather than supply one.

Four of the six elements measure the flow of information between markets, and the economics says they should find something to measure. The Grossman–Stiglitz argument [22] implies that informationally efficient markets are impossible, so a residue of costly, privately held, unequally distributed information must persist; transfer entropy is the programme's instrument for tracing the flow of exactly that residue, directionally and across scales. A reading caution applies throughout. These markets are adaptive and heterogeneous [28], so a structure measured in one regime need not persist into the next, and several elements below report exactly such a dissolution or a knife-edge. We treat those as findings, not as failures of the apparatus.

7.1 NAMH: adaptive market networks

Question. Do equity markets form a directed information network whose broadcasters and receivers can be identified, and is that structure stable as the network adapts over two decades? This is the network reading of the adaptive markets hypothesis [28]: efficiency as an evolving, relational property rather than a fixed one. The economic stake is whether information leadership in the global equity system is persistent and locatable, or whether it migrates as participants adapt; an answer that holds across twenty years of windows is an answer about structure, while one that dissolves is itself a statement about adaptation.

Primitives. Long-memory estimation (a Hurst exponent per market), transfer entropy for the directed edges, IAAFT surrogates and false-discovery control for edge significance, and network formation with directed centrality (out-strength, in-strength, PageRank, Katz, eigenvector) read on the resulting graph.

Result. The FRONTIERS Paper I treatment, a seventeen-page study over twenty-eight assets from 2001 to 2026, finds that the United States broadcasts and Japan receives. The United States leads on out-strength in 9 of 20 windows annually and 26 of 79 quarterly; Japan receives, leading on in-strength in 12 of 20 windows, on PageRank in 10 of 20, and on Katz centrality in 8 of 20 [6, 9]. On the engine, the NAMH transfer-entropy estimators run through the paper's own `namh` package: the window-mean transfer entropy is -0.22322165, inside its pre-registered band, and reproduces against the cached run to a maximum absolute deviation of 1.55×10^{-15}.

Honest status. Two honesty markers belong on the record. First, the broadcaster/receiver reading carries an exception, stated rather than smoothed: the quarterly Katz leader is Argentina with 22 edges, narrowly over Japan's 21. Second, on the engine the false-discovery-controlled NAMH network is degenerate: 0 of 552 candidate edges survive the gate, and we report that as a degenerate network rather than relaxing the threshold to manufacture structure. The headline ranks above are magnitude-ordered centralities; the FDR result is a separate, stricter question, and the two are kept distinct.

7.2 SOCH: scale-ordered contagion

Question. Does cross-border contagion have a spectral signature, an ordering across time scales that reflects how heterogeneous participants adapt to information at different horizons? SOCH formalises that ordering and asks whether it holds in the data. The economic motivation is that a high-frequency trader and a pension fund respond to the same shock on different clocks, so contagion that looks alike in aggregate may be ordered by scale once it is decomposed; recovering that order distinguishes fast, mechanical transmission from slow, fundamental adjustment.

Primitives. MODWT to resolve a relationship by scale, transfer entropy within each scale band for the directed contagion profile, IAAFT surrogates for significance, and a spectral-density construction that the formal layer machine-checks.

Result. On the engine the scale-resolved contagion profile from the United States to India aggregates to 0.0391 over four scales, peaking at scale `d4`, computed through the published `sochcontagion` package [7, 8]. The robustness badge holds the baseline cell ($\tau = 0.05$, four filter orders) across all 28 market pairs and returns an overall pass rate of 0.9911 (111 of 112 grid cells), under a seeded null. The spectral theory is supported by a formal layer: of twelve Lean 4 declarations, eleven are machine-accepted, including the unconditional closure of the product-Lorentzian spectral-density proposition, $\|H(\omega)\|^2 = S(\omega)$.

Honest status. The framework's three pre-registered significance tests do not all clear. SOCH-A is significant at $p = 0.042$ and SOCH-B holds 28/28, but **SOCH-C is not significant, at** $p = 0.105$, and we treat that as an immutable fact of the record rather than a result to be managed. The formal layer is likewise reported as it stands: the single unproved declaration, the analytic half of the peak-location lemma, honestly carries a `sorry` (sorry-count 1/12), and a `sorry` is never reported as proved.

7.3 Contagion channels

Question. When contagion crosses a border, through what does it travel? The framework identifies and attributes cross-border transmission to structural channels (trade, finance, and others) rather than measuring only that transmission occurred. The distinction matters for policy: a shock that propagates through the trade channel calls for a different response than one carried by financial linkages, and an attribution that does not name the channel leaves the question that policy actually asks unanswered.

Primitives. Channel attribution as the central primitive, decomposing a transmission measure across structural channels, supported by the network and connectedness machinery that supplies the transmission edges and by episode partitioning across crises.

Result. Across eight crisis episodes the framework attributes the dominant share of global-financial-crisis contagion to the trade channel at 27.9%, with an interval of [26.1, 29.8] [10, 11]. On the engine the attribution runs through the published `contagionchannels` package and reproduces the trade share to 0.000 percentage points against the archived decomposition. This is the cleanest reproduction in the programme: the engine's live number equals the paper's to the reported precision.

Honest status. The trade share is a contested figure across method and partition, and is carried as such (see WaveQTE below); the engine's contribution is to make the canonical 27.9% re-runnable to the digit, not to adjudicate the contest.

7.4 WaveQTE: wavelet quantile transfer

Question. How does tail risk transfer between markets across both time scale and quantile, and how much of an episode's contagion does a wavelet-quantile reading attribute to a given channel? The economic interest is in the tails specifically: ordinary co-movement and crisis co-movement are different objects, and a method resolved by quantile asks how transmission behaves when it matters most rather than on average.

Primitives. MODWT for the scale resolution and quantile transfer estimation for the tail-dependent directed flow, with the same surrogate and network primitives downstream.

Result. A wavelet-quantile reading attributes 50.0% of the relevant episode's contagion where the structural channel decomposition of Section 7.3 attributes 27.9%. On the engine, the `wqte` method returns a United-States-to-India quantile transfer of 0.039 at $\tau = 0.05$, an aggregate that sits inside its pre-registered band.

Honest status. The 50.0%-versus-27.9% gap was investigated and resolved, and the resolution is itself the finding: **it is a method-and-partition divergence, not a bug.** The two figures come from different estimators (a wavelet-quantile activation-scoring reading against a structural instrumental-variables decomposition) over different episode partitions, so they are measuring related but distinct quantities and are expected to differ. One further honesty marker: the framework's packaged Stage-1 estimator is structurally unavailable inside the deployed image, so the engine's `wqte` is an honest MODWT-plus-quantile-regression reimplementation, declared as such, rather than a call into the original Stage-1 code. It is banded like any other method and is not presented as the packaged estimator.

7.5 Commodity systems

Question. How does the commodity complex fragment and re-form across crises, and which episode leaves it most fragmented? The object is the network of commodities itself, read as a system that reorganises under stress. The economic content is the integration of the commodity complex: a complex that fragments under a crisis offers diversification exactly when other markets are co-moving, so locating the deepest fragmentation locates the episode in which the complex behaved least like a single asset.

Primitives. Network formation over the commodity cross-section, community detection to recover the cluster structure, and a fragmentation measure read across crisis episodes.

Result. Over twenty-three commodities from 1991 to 2025, the Subprime crisis leaves the complex most fragmented, with a fragmentation coefficient of magnitude $\|\mathrm{FC}\| = 0.290$ and a partition into five communities. The finding is that the deepest fragmentation of the commodity network in this sample sits at the Subprime episode rather than at a later crisis.

Honest status. The fragmentation magnitude and community count are the verified figures; finer claims about which commodities migrate between communities are governed by the published study's own community-structure record and are not restated here beyond what the verified record holds. Where a cross-episode comparison would need per-episode fragmentation coefficients beyond the Subprime magnitude, figures the verified record does not carry, we leave them unstated rather than assert them.

7.6 MCPFM: protocol-mediated systemic risk

Question. Can a protocol-mediated, multi-scale network model of the financial system forecast systemic-risk episodes, and does that forecasting ability survive a pre-registered re-evaluation? This is the element where the programme tests one of its own models against an out-of-sample, honestly scored benchmark. The economic claim under test is strong, that network structure carries predictive content about systemic stress, and the value of testing it under pre-registration is precisely that the apparatus cannot quietly retreat to the sample and episode where the claim looks best.

Primitives. The full network and connectedness stack feeding a multi-scale systemic-risk construction, with ROC evaluation as the primitive that scores the forecasts, here read through the area under the curve and a formal test of predictive superiority.

Result. In its first evaluation the model returns an area under the ROC curve of 0.581 (DeLong $p = 0.030$) over the full sample, rising to 0.915 on the United States during the COVID episode [5]. A *pre-registered* second-version re-evaluation then returns an area under the curve of 0.470 with a Hansen superior-predictive-ability $p = 0.031$, and is reported openly.

Honest status. This is the programme's sharpest negative. **The pre-registered v2 re-evaluation returns AUC** 0.470, below the 0.5 chance line, and we report it as such rather than retreating to the favourable v1 figures. The pre-registration is what gives the negative its force: the test was fixed before the run, so a result below chance is a result, not a discard. The reproduction status of the systemic-risk index is carried as honest-pending, consistent with the rest of the programme's reproduce discipline.

7.7 Cross-framework consistency

Read together, the six elements are not six tools but six configurations of one substrate, and that is the claim the engine makes good. The same long-memory estimator that gives NAMH its per-market Hurst exponents is the estimator the engine bands as `namh_hurst`; the same KSG/Frenzel–Pompe transfer-entropy kernel supplies NAMH's directed edges, SOCH's within-scale contagion, and the engine's headline United-States-to-Japan flow of 0.154234; the same MODWT resolves SOCH and WaveQTE by scale; the same IAAFT surrogates and false-discovery control gate significance across NAMH, SOCH, and WaveQTE; the same network and community-detection machinery recovers structure for NAMH, the commodity complex, and MCPFM; and the same channel-attribution primitive that the contagion-channels element reproduces to 0.000 percentage points is the one whose method-dependence the WaveQTE divergence illustrates. Verifying a primitive therefore verifies it for every element that instantiates it, which is the

Element	P1	P2	P3	P4	P5	P6	P7	P8	Headline (verified)
NAMH	•		•	•	•	•			US out-strength 9/20
SOCH		•	•						SOCH-C $p = 0.105$ (n.s.)
contagion-channels		•	•		•		•		trade 27.9%
WaveQTE		•	•	•					`wqte` 0.039
commodity			•		•	•			$\|FC\|$ 0.290
MCPFM		•				•		•	v2 AUC 0.470

Table 6: The six elements against the eight substrate primitives: P1 long memory, P2 MODWT wavelets, P3 transfer entropy, P4 surrogates, P5 community detection, P6 network connectedness, P7 channel attribution, P8 ROC/DeLong. A marker indicates the element instantiates that primitive (the principal-primitive view of Figure 3); the final column gives one headline verified figure per element, positive and negative alike.

economy the programme framing is meant to buy: twenty-six checked methods carrying six frameworks, rather than six codebases asserting their own numbers in parallel.

Table 6 lays the six elements against the eight primitives. The filled columns for transfer entropy and network connectedness show how few independent estimators carry the whole programme.

8 Deployment and Operations

A research engine is only as reproducible as the procedure that ships it and the discipline that keeps it running. This section records how the kernel reaches Cloud Run, the isolation and credit posture it runs under, and the two standing operations that keep the system live without supervision: the nightly self-check loop and the systemic-risk feed. Throughout, the governing constraint is the same as in the architecture – we expose nothing whose cost or blast radius we cannot bound in advance.

8.1 Deploying the kernel to Cloud Run

The kernel is deployed by `cloudrun/deploy.sh`, which builds a temporary context and runs `gcloud run deploy --source`. The build step exists because the one bundled dataset, `G20.xlsx`, lives outside the engine directory and Docker cannot `COPY` from outside its context; the script stages the server, the R analyses, the web assets, and the three published method packages (`sochcontagion`, `contagionchannels`, `namh`) into a scratch directory alongside the dataset, then deploys that. The image is defined by `cloudrun/Dockerfile` from `rocker/r-ver:4.4.1`.

The R stack drives most of the image. The `igraph` binary links against GLPK, libxml2 and GMP, so the Dockerfile installs the system libraries `libglpk40`, `libxml2` and `libgmp10` *before* the R package install, both so the post-install load check succeeds and because the method needs them at runtime; the remaining R packages are pure-R or Rcpp and need no system dependencies. The three published packages are installed from their bundled tarballs, so a method such as channel_attribution or namh_te runs the paper's own code, not a reimplementation, and the build fails loudly if any package or its bundled data is missing. Node 20 is added last; the server itself is zero-dependency.

Two deployment facts carry operational weight. First, the model API key is injected, never baked in. The script resolves `GOOGLE_API_KEY` from the environment or from a `.env.local` one level above the repository and passes it via `--set-env-vars`; it is never printed and never committed. If the key is absent the script prints an explicit warning and ships a revision on which `/api/chat` is simply disabled – a degraded but honest service, not a broken one. A revision

accidentally shipped without the key can be repaired in place with `gcloud run services update --update-env-vars`, no rebuild required.

Second, the deploy flags encode the cost posture. The service is deployed with `--timeout 300`, `--min-instances 0`, `--max-instances 2` and `--concurrency 16`, alongside the runtime caps `COMPUTE_TIMEOUT_S=90`, `MAX_CONCURRENT=8` and `MAX_LLM_PER_DAY=400`. The `--min-instances 0` is the scale-to-zero economics of Section 2.1: between requests the service costs nothing, which is appropriate for occasional, bursty research traffic and is what makes a publicly reachable compute endpoint affordable to operate indefinitely. The remaining caps are the credit and abuse controls of Section 8.3.

8.2 Isolation and the Tier-0 hardening posture

On Cloud Run, gVisor is the isolation boundary around the container, so the server's `bwrap` path is skipped by auto-detection and analyses run under the `timeout` fallback (Section 2.6); the container holds no secrets beyond the injected key and the G20 panel is the only bundled dataset. We are explicit that this is `timeout` mode, not a network-isolated `bwrap` sandbox – the live `/health` says so – and the no-arbitrary-code guarantee therefore rests on the parameterised-only registry (Invariant 1), which holds in either mode.

On top of that boundary sits a layer of hardening whose purpose is to make a public launch safe against credit exhaustion and abuse rather than against code injection. The kernel applies, before any model call or spawn: a global per-instance concurrency ceiling (`MAX_CONCURRENT`, shedding excess with `503`); a per-IP per-minute limiter and a per-IP daily quota on the paid routes; a 64 KB request-body cap and a message-length cap that reject oversized or over-long inputs with `413` before any Gemini call is made; a slowloris guard on request and header timeouts; and a stale-cache fallback that serves the last good result, clearly flagged, when a live run fails rather than returning an error. Adversarial load testing of this posture – per-IP bursts, multi-IP fan-out, oversized bodies, over-length inputs, method-injection probes, and budget exhaustion – produced no uncoded error and no `500`; every rejection was a coded JSON response. The binding financial control among these is the daily model budget, treated next.

8.3 The cost bound on the AI layer

The AI routes are the only paid component, and their spend is bounded by a hard, verifiable ceiling rather than by a pricing estimate. Each instance enforces a global per-day cap of `MAX_LLM_PER_DAY = 400` requests that reach the paid stage, reset at UTC midnight, on top of the per-IP limits. Because Cloud Run is deployed with `--max-instances 2`, the fleet-wide ceiling is the product:

$$\text{paid turns/day} \;\le\; \underbrace{400}_{\text{per instance}} \;\times\; \underbrace{2}_{\text{max instances}} \;=\; 800.$$

At most 800 paid turns per day can occur regardless of how many distinct IPs attack the service. A "turn" is one chat or research request that reaches the paid stage; over the cap, the route returns `503 daily_capacity`, never a fabricated answer. This bound is what makes the launch safe: it is independent of attacker IP count, it is enforced before the Gemini call so an over-cap request spends nothing, and it is tunable through the environment. The application-level caps are the active backstop; a billing-account budget alert is an additional financial tripwire to be set by the project owner.

8.4 The governed cloud-capability layer

The cost bound of the previous subsection generalises into a small framework, because the language-model analyst is no longer the only managed-cloud surface the engine can reach. A governed capability layer in `server/upgrade-capabilities.mjs` exposes a fixed menu of paid

Google Cloud services through one route, `POST /api/upgrade/run`, whose body names a capability and a set of typed parameters. The route reuses the registry discipline of Section 2.5: it matches the name against a fixed four-entry table, refuses anything else with a coded error, and is fetch-only, so it adds no new execution surface and the parameterised-only guard is unchanged.

Each capability is disabled unless its own flag, `CE_CAP_<NAME>`, is set explicitly, so the resting posture is least-privilege and off. Every paid call then passes a fixed gate in order, the flag, the typed-parameter check, a prerequisite check where one applies, and last a spend ceiling, before any request leaves the engine. A rejection at any stage returns a coded capacity or argument error and spends nothing, never a fabricated answer, exactly as the analyst cost bound does. The ceiling is one reusable governor, `capBudget`, in the same guards module as the analyst budget; it keeps a per-UTC-day counter for each named capability, billed in calls or, for embeddings, in tokens, and it reserves the units *before* the call so that an over-cap request is refused rather than charged. It generalises the single `MAX_LLM_PER_DAY` counter of Section 8.3 to a family of named counters, and like the core budget of Section 4.1 it is the governor a saturable resource must ship with before it is reachable. We confirmed the ceiling on the live service: with the Gemini-on-Vertex flash cap set to one for the test, the second request of the day returned the coded capacity error rather than a result, and the cap was then restored.

Capability	Vertex surface	Daily ceiling	Status
Embeddings	`text-embedding-004`	2,000,000 tokens	live
Grounded search	Vertex AI Search	200 queries	live
Gemini-on-Vertex	`aiplatform` generate	300 flash / 100 pro	live
Batch prediction	`batchPredictionJobs`	1 batch, $\leq$ 50,000 items	live

Table 7: The four governed cloud capabilities, each behind a default-off flag with a per-UTC-day spend ceiling that trips before the paid call. All four are live end-to-end on the deployed service; the batch path reached completion once a single managed-dependency role grant was in place.

All four are live end-to-end. The grounded retrieval is worth a sentence of its own, because it is the one that touches citation integrity: it queries a Vertex AI Search datastore built over the engine's own literature warehouse, sixty-three documents imported from the `literature.papers` table, and it returns real document identifiers, so a grounded answer points to a passage that was actually retrieved rather than to an invented reference. This is a different mechanism from the web-search grounding of the research route in Section 5.2; here the corpus is our own. The fourth capability, batch prediction, stages its input, submits a real asynchronous job, and writes the resulting embeddings back to storage on completion. Reaching completion required one managed dependency to be satisfied: the embedding batch is routed through a managed delegation account that needed a single role grant to run on the project's behalf. With that grant in place the path runs end-to-end, verified by a submitted job that completed and returned its embeddings.

Two properties close the layer. A secret never leaves the boundary: a scan gate refuses to commit or deploy any file carrying a secret-shaped value, reporting only a redacted fingerprint, and it matches values rather than environment-variable names so the engine's own key-reading code does not trip it; the route logs the capability name and the coded outcome but never the parameters and never a token, and the cloud credential is minted at the infrastructure boundary and used only as a bearer header. And the layer is reversible: clearing a capability's flag returns it to a disabled state, and the retrieval datastore can be deleted to return its cost to zero. The failable evaluation `test/upgrade.test.mjs` exercises every gate, default-off, the typed-parameter check, the ceiling trip with no token minted, the prerequisite hole with no spend, and the secret-scan blocking then clearing; six governance dossiers in `upgrade/` carry the inventory, the cost model, the threat model, an activation ledger, a reverification report, and the gate.

8.5 The SRI live feed as a standing operation

The systemic-risk feed is the engine's one continuously-updating product, and it is built to run unattended. A daily Cloud Scheduler tick invokes `POST /api/sri/cron-tick`; the kernel pulls the latest market closes from a public source, appends them to the stored returns panel, recomputes the index in a net-isolated step, and writes the new value to the `systemic_risk.daily` store. The recomputation reuses the same KSG transfer-entropy estimator as `ksg_te` (no surrogates), over the most-recent rolling window of the panel, across every active directed pair; the index is the mean directed transfer entropy, a system-wide nonlinear information-flow proxy where higher means more interconnected. It is deliberately a connectivity index and is not the MCPFM validation construct; the two are not conflated.

The operation is idempotent and self-healing. The trusted Node layer fetches the data and reads or writes the data store, while the R step only ever sees an injected panel and never the network, preserving net-isolation. A missing day is skipped honestly rather than fabricated, and the panel's composition heals the gap on the next run. The tick also exposes a dry mode that exercises the full read-and-compute path with no writes, which serves as a health check. The feed handles markets that drop out of the panel by excluding only their own directed pairs at the affected date, so a date is a consistent constituent set rather than a stale mix: the first value, on 2026-03-18, was an SRI of 0.00849 over 306 directed pairs, and the 2026-06-05 value was 0.00709 over 272 pairs, the smaller pair count reflecting markets without data in that window. One operational rule governs the schedule: the tick is never fired manually during the US trading session, because the fixed schedule is itself the guard against computing on a partial bar.

8.6 The nightly self-check loop

The kernel and feed are built to run unsupervised: the daily systemic-risk tick already runs on a managed scheduler, and a nightly loop, installed in cron, re-establishes the system's invariants and fails loudly when one breaks. The loop re-arms the tower if its port is closed without ever killing a running job, runs the full evaluation suite in place to refresh `evals.json`, refreshes the measured-state and claims ledgers (a regression exits non-zero; a red result becomes a contested pair and nothing is deleted), and finally, at low priority and doing zero compute, renders the academic run report from that night's `evals.json` verbatim and publishes it to the gallery. The report and gallery steps recompute nothing, so the reporting layer can never re-trigger heavy work. The discipline of the loop – real compute under pre-registered failable bands, never hand-edited artefacts, honest negatives preserved – is the operational embodiment of the verification posture that the rest of the manuscript develops; its own wiring is itself a failable evaluation row.

9 Worked Examples

The preceding sections describe the engine's planes, its guards, and its verification layer in the abstract. This section makes them concrete with four end-to-end traces, each a real path that the system runs: a synchronous transfer-entropy call, an exact reproduction of a published result, a GPU re-run of the workload that once hung the workstation, and one nightly tick of the live systemic-risk feed. Each trace is chosen because it exercises a property argued elsewhere in the paper, and each ends in a number that traces to the verified-results record.

9.1 A synchronous request and its response

The most common path is a single method call returning structured data. We follow the directed transfer entropy from the United States to Japan, the edge that the `ksg_te` evaluation row bands.

Example 4 (KSG transfer entropy, request to response)**.** A client posts a method name and a validated parameter object; no code is sent.

```
POST /api/compute/run
{ "method": "ksg_te", "params": { "series": ["USA", "Japan"] } }
```

The request crosses five stages before a number comes back.

1. **HTTP and rate guard.** The kernel accepts the request on `/api/compute/run` under its per-minute rate limit.
2. **Registry guard.** The method name is looked up in the fixed registry. ksg_te resolves; an unknown name is a clean rejection. The parameter object is validated against the method's schema. *No user input reaches a shell, because the registry admits no user code in the first place.*
3. **Dispatch.** ksg_te is one of the heavy asynchronous methods, so the work is handed to the always-on workstation job server rather than run inline; the call returns a job identifier and is polled to completion (here `job_20260612_92fa3f31`).
4. **Sandboxed R spawn.** The runner script for the method is spawned under the sandbox, reads the stamped `G20.xlsx` panel, computes the Kraskov–Stögbauer–Grassberger nearest-neighbour transfer entropy [25] in the Frenzel–Pompe form [21], and assesses significance against IAAFT surrogates [34].
5. **Single-JSON stdout.** The script writes exactly one JSON object to standard output; the kernel parses it and returns the result.

The response carries the directed edge and the panel-level summary:

```
{ "ok": true,
  "result": { "edges": [ { "from": "USA", "to": "Japan", "te": 0.154234 } ],
              "n_pairs": 306, "n_significant": 108 },
  "ms": 105081, "cached": false }
```

The headline edge is **0.154234**; across the 18-market panel the run scores **306 directed pairs**, of which **108** are significant. This is the exact value the ksg_te band $[0.150, 0.158]$ checks in Section 6.1.

9.2 An exact reproduction

The second trace re-runs a published estimator and matches its output to machine precision. This is the property a reproduction target exists to buy: not a number close to the paper's, but the paper's number, produced by the paper's own code inside the sandbox.

Example 5 (Reproducing the published contagion channel share)**.** The channel_attribution method runs the published `contagionchannels` package, not a reimplementation, and reports the global-financial-crisis channel decomposition.

```
POST /api/compute/run
{ "method": "channel_attribution", "params": { "episodes": ["GFC"] } }
```

The engine returns the trade-channel share for the crisis window:

```
{ "ok": true,
  "result": { "episodes": [ { "episode": "GFC",
                              "dominant": "Trade", "trade": 27.9 } ] },
  "ms": 25060 }
```

The trade share is **27.9%** and the dominant channel is Trade, reproducing the paper's published Table 5 to **0.000 pp** [10]. The evaluation row checks both the share band $[27.8, 28.0]$ and the dominant-channel label, so a drift in either fails the row.

A second, even tighter reproduction lives one layer down, in the dedicated delta-verifier namh_reproduce. It re-runs the published `namh` package estimators and reports the maximum absolute deviation from the archived output, cell by cell. On the workstation the rolling Hurst panel and the per-window raw transfer entropy reproduce to a maximum absolute deviation of 1.55×10^{-15}, well inside the green threshold of 10^{-8}; the false-discovery network is held *amber* by design, with 0 of 552 edges retained, because that null is the published result and a green there would be a silent over-claim. A deterministic estimator earns a green only on an exact match; a stochastic surrogate stage, whose generator was unseeded in the original run, is reported amber rather than dressed as exact. Reproduction is graded, not asserted.

9.3 A GPU re-run, bit-exact to the CPU estimator

The third trace is the workload that once saturated all twenty-two logical cores of the workstation: the full directed transfer-entropy panel with surrogate significance. Section 6.3 records the two lessons that made it tractable – one governed core budget with pinned linear algebra (L11), and a bit-exact GPU offload of the nearest-neighbour search (L12, L13). This trace shows the offload at full panel scale.

Example 6 (The full panel on one GPU)**.** The same ksg_te request, but issued over the entire 18-market panel with 99 surrogates per pair, dispatches to the GPU when a CUDA device is present and falls back to the governed CPU otherwise (the default is automatic; `CE_GPU=0` forces the CPU path).

```
# 306 directed pairs x 99 IAAFT surrogates = 30,600 neighbour searches
POST /api/compute/run { "method": "ksg_te", "params": { "dataset": "g20" } }
```

The neighbour search runs in two CUDA kernels; the surrogate generation runs on the host, because that stage is unseeded and its p-values are non-reproducible in either path. The exact workload that previously hung the box completed in **35.6 minutes at 100% of one core**, peak **388 MB**, scoring **106 of 306** pairs significant (the significant-edge count moves by a pair or two between runs, against the 108 of the synchronous eval row, precisely because the surrogate draw is unseeded; the observed transfer entropy the gate checks does not move), with USA-to-Japan returning **0.154234** exactly and Japan the dominant information sink. The governing comparison is not speed but agreement: the GPU's observed transfer entropy equals the CPU estimator to 6.7×10^{-16}, inside the pre-registered bit-exactness gate of 7×10^{-16}, and the observed transfer entropy is the only quantity the evaluations gate. The GPU-default therefore never moves a published number; it only frees the processor.

On a smaller, controlled comparison (12 pairs, 99 surrogates) the GPU path completed in 82 s on roughly one core against 289 s on roughly seven governed cores, a 3.5× speed-up that simultaneously returns the rest of the machine. The bar held throughout is the one stated as an invariant elsewhere: the device must *equal* the CPU estimator on the gated quantity, not approximate it.

9.4 A nightly tick of the systemic-risk feed

The fourth trace is the self-sustaining one: the daily append that keeps the live systemic-risk index current without supervision. It runs ahead of the nightly verification loop of Section 6.2, at 06:00 UTC, and is built to be safe to repeat and honest when its inputs are not ready.

Example 7 (One idempotent SRI append)**.** A scheduled trigger posts to the kernel's tick endpoint, which pulls the latest closes, appends one row to the rolling panel, recomputes the index over a net-isolated computation, and persists it.

```
POST /api/sri/cron-tick # nightly; ?dry=1 is a health check that writes nothing
```

For the 2026-06-05 close the tick produced an index of **0.00709** over **272 directed pairs**. The append is idempotent: re-firing the tick for a date already present writes nothing rather than double-counting. It is self-healing: when the upstream feed serves a null close for hours after a session, the finite-close guard makes the tick skip that day honestly, and the next day's composition heals the gap rather than back-filling a guessed value. And it is observable without side effects: a dry-mode call recomputes and reports but does not write, so the feed's health can be probed at any time.

The two systemic-risk numbers in this paper bracket the feed's history and trace to the same record: the first honest static-panel point, **0.00849** over 306 directed pairs on 2026-03-18, which the sri_daily evaluation row bands to $[0.00848, 0.00850]$; and this live tick, **0.00709** over 272 pairs on 2026-06-05. The pair count differs because the live panel carries the markets with a finite close on the day, and the index recomputes in full from the stored panel each night rather than incrementing a cached figure – the same re-runnable-from-source discipline that the synchronous, reproduction, and GPU traces above each demonstrate in their own way.

10 Design principles, limitations, and outlook

The engine is deliberately small, and we intend to keep it so while widening what it can certify. This closing section distils the principles that the preceding sections established, states the limitations plainly, and describes the self-renewing programme we are building towards. The throughline is unchanged from the first version of the engine: a research result should be a re-runnable, self-checking object, and the infrastructure that makes it so is worth building once, carefully, and maintaining in the open.

10.1 The design principles, distilled

Five principles carry the design. Each was learned in operation, several the hard way, and each is stated here in the form the project applies to itself.

Principle 9 (Parameterised-only registry)**.** A request names a method and schema-validated parameters; it never supplies code. The registry is therefore the primary guard against arbitrary-code execution, and the operating-system sandbox is defence in depth on top of it, not the load-bearing control.

Principle 10 (A failable test per method)**.** A method is not in the catalogue until it ships with a pre-registered, failable test of its own output. Holes in coverage are marked, never filled. The registered preference is explicit: we would rather have twenty-six checked methods than fifty asserted ones.

Principle 11 (Honest negatives are results)**.** A result that does not clear its test, or that contradicts an earlier finding, is reported as such and treated as an immutable fact of the record. The not-significant SOCH-C test ($p = 0.105$), the degenerate false-discovery-controlled NAMH network (0/552), and the below-chance MCPFM v2 re-evaluation (AUC 0.470) are findings, not blemishes to be managed away.

The remaining two principles are properties the system maintains rather than preferences it expresses, so we state them as invariants.

Invariant 8 (Governed determinism)**.** Worker count never changes a number. A single governed core budget, clamped last by a server-set hard ceiling, with linear-algebra threading pinned to one thread per spawned process, makes every result independent of how many workers run it. The transfer-entropy loops are order-free, so this is determinism-safe, and the GPU offload of the same computation is held bit-exact to the CPU estimator on the only gated quantity, the observed transfer entropy, to within machine epsilon.

Invariant 9 (Machine-written ledgers are never hand-edited)**.** The single evaluation artefact, the state ledger's measured block, and the claims record are written by their generators and never by hand. A regression turns the public dashboard red rather than passing silently; the state ledger is loud about drift; and the claims record never deletes, so a result that turns red becomes a contested pair rather than a quiet disappearance.

These five are not independent. The failable-test principle is what makes the honest-negative principle enforceable; the never-hand-edit invariant is what makes both credible to a reader who was not in the room. Together they are the sense in which the engine's reliability is a property of its process rather than a promise from its authors.

10.2 Limitations

We state the limitations plainly, because the discipline that reports honest negatives in the elements applies equally to the engine that carries them.

Reported degeneracies and negatives. Some of the programme's headline tests do not clear, and we have not hidden this. The false-discovery-controlled NAMH network is degenerate, retaining 0 of 552 candidate edges; the engine's NAMH centralities are magnitude-ordered and are not a claim that the FDR-gated network is non-empty. The pre-registered MCPFM v2 re-evaluation returns an area under the ROC curve of 0.470, below chance. The SOCH-C significance test is not significant at $p = 0.105$. None of these is a bug to be fixed; each is a fact of the record.

Infrastructure constraints. The heavy asynchronous estimators run on a single workstation tower, so the engine's throughput on the four tower methods is bounded by one machine and its one GPU; a second compute node is a pre-registered future step, not a present capability. The natural-language layer is rate-limited and budget-capped: over its daily cap it returns an explicit capacity error with a reset hint, never a fabricated answer, which is the right failure but is a hard ceiling on interactive use. The formal layer is partial: of twelve Lean 4 declarations, eleven are machine-accepted and one, the analytic half of the peak-location lemma, honestly carries a `sorry`; the spectral theory is closed where the proposition $\|H(\omega)\|^2 = S(\omega)$ is concerned, but the peak-location chain is not yet complete.

Scope. The engine certifies what its twenty-six methods cover and no more. A question outside the catalogue has no banded answer, by design, and a figure outside the verified record is marked for verification rather than asserted. This is a deliberate narrowness, but it is a narrowness, and the honest description of the engine is that it makes a fixed catalogue re-runnable, not that it makes financial econometrics in general reproducible.

10.3 Towards a self-renewing programme

The direction of travel is to make the engine adaptive in the same disciplined sense the markets it studies are adaptive: a system that learns from its own operation without ever loosening the gate that keeps it honest.

The nightly loop as a learning system. The engine's daily systemic-risk tick already runs unattended on a managed scheduler. A reboot-surviving nightly loop, now installed in cron, re-arms the tower, runs the full public suite as one genuine execution with the asynchronous rows submitted as real jobs, refreshes the state ledger and appends any drift, refreshes the claims record without ever deleting, and renders and publishes an academic report of the night's run. Crucially, the loop's own wiring is itself a failable evaluation: the system that checks the engine

is checked by the same kind of pre-registered test it applies to everything else. The next step is to treat the sequence of nightly runs as a learning record, so that drift over time becomes a first-class signal rather than a per-night log line.

Pre-registered, PI-gated gap proposals. A gap-proposal pipeline reads the engine's own usage and coverage and proposes candidate additions to the catalogue. It is pre-registered and never auto-deploys: a proposal is a review document for the principal investigator, not a change to the live registry. When the pipeline ran end-to-end and its rule found no qualifying gap, that no-proposal outcome was recorded as a first-class result, in the same spirit as a not-significant test. The design intent is a catalogue that can grow on evidence while every addition still passes through the same human gate and the same failable-test requirement.

More elements, more formal coverage. Two expansions are in view on the same terms. First, more elements may join the programme, each only with a failable evaluation and each instantiating the shared substrate, so that the marginal cost of a new framework is the cost of its configuration and its test, not a new codebase. Second, the formal layer can grow: closing the remaining `sorry` in the SOCH peak-location chain, and extending machine-checked proof to results beyond SOCH, would widen the set of claims the engine can certify at the level of a checked proof rather than a banded number.

The self-renewing vision. Taken together, these point at a programme that renews itself: a fixed, sandboxed engine; a shared substrate that new elements configure rather than reinvent; a nightly loop that learns from its own record; and a gap pipeline that proposes growth under a standing human gate. We do not claim this is finished, and we have stated where it is partial. We claim only that the parts are built, that they hold to the standard we have described, and that the standard, not any single number, is the contribution we are most willing to defend.

A The method registry

This appendix reproduces the canonical registry of **twenty-six methods**, grouped into six families, exactly as the committed evaluation runner `econstellar/evals/run-evals.mjs` carries them. For each method we give the verified figure recorded by the most recent full suite (2026-06-12), the pre-registered acceptance band or check, and the methodological basis. The bands are fixed before the run and are never hand-edited; `evals.json` is always the output of one genuine run (see Section A and the integrity invariants in Appendix C). A method is not in this table until it ships with a failable test of its own output.

Method	Value (2026-06-12)	Band / check	Basis
I. Stationarity, cointegration and panels			
`unit_root`	India ADF -49.18	ADF stat $\in [-55, -45]$	ADF+KPSS (`urca`, `tseries`)
`live_unit_root`	verdict	levels non-stationary *and* returns stationary	live ADF gate
`panel_unit_root`	IPS -77.26	IPS < -10 *and* LLC < -10 (LLC -51.79)	IPS / LLC panel
`vecm`	rank 3	Johansen rank $= 3$ (trace $7265.97 \to 1851.19$)	`urca` Johansen
`granger`	6 edges	$n_{\text{edges}} = 6$ (USA out-degree 3)	Granger causality

continued on next page

Method	Value	Band / check	Basis
II. Volatility, long memory and spectra			
`garch`	$\alpha+\beta$ 0.991	persistence $\in [0.95, 1.0]$	GARCH(1,1) (`tseries`)
`dfa_hurst`	H 0.542	$H \in [0.45, 0.65]$	DFA
`namh_hurst`	Gold H 0.490	$H_{\text{mean}} \in [0.4889, 0.4909]$	`namh` package
`wavelet`	47.07	scale-1 % of total $\in [35, 60]$	MODWT variance (`waveslim`)
`wavelet_coherence`	0.249	mean coherence $\in [0.2, 0.3]$, peak scale `d6`	MODWT coherence
III. Information flow (transfer entropy)			
`ksg_te`	USA→Japan 0.154234	TE $\in [0.150, 0.158]$	KSG / Frenzel–Pompe
`ksg_robustness`	mean Spearman 0.7282	headline edge rank = 1 across all 8 grids *and* mean Spearman $\in [0.6, 0.85]$	KSG nuisance grid
`namh_te`	−0.22322165	window mean $\in [-0.2242, -0.2222]$	`namh` package
`wqte`	0.0391	aggregate QTE $\in [0.02, 0.06]$	wavelet-quantile (`waveslim`+`quantreg`)
`soch_profile`	0.0391	forward $\in [0.03, 0.05]$, peak scale `d4`	`sochcontagion`
IV. Connectedness and networks			
`connectedness`	TCI 30.25	TCI $\in [25, 35]$	Diebold–Yilmaz
`spillover_rolling`	mean TCI 28.39	mean rolling TCI $\in [20, 35]$	rolling Diebold–Yilmaz
`rolling_dcc`	0.2295	India–USA mean corr $\in [0.15, 0.31]$	DCC
`network`	density 0.5667	6 nodes, density $\in (0, 1]$	`igraph` surface
`var_irf`	max root 0.7053	companion max root $\in [0.01, 0.999]$	VAR / IRF (`vars`)
`quantile_var`	net +0.7006	top tail driver = USA, net > 0	quantile VAR (`quantreg`)
V. Contagion channels and systemic risk			
`channel_attribution`	GFC trade 27.9%	trade share $\in [27.8, 28.0]$ *and* dominant channel = trade	channel attribution [10]
`sri_daily`	0.00849	SRI $\in [0.00848, 0.00850]$	daily systemic-risk index
VI. Reproduction and formal verification			
`namh_reproduce`	Δ 1.55×10^{-15}	Hurst & TE panels green, $\max\lvert\Delta\rvert \leq 10^{-8}$, $\geq$ 400 compared, 0 NA-mismatch; FDR amber, 0 edges	`namh` Δ-verifier
`namh_pipeline`	−0.22322165	FDR 0/552, network degenerate, window mean $\in$ band, seed 42, RNG L'Ecuyer-CMRG	`namh` end-to-end (async)
`soch_robustness`	pass-rate 0.9911	baseline $\tau = 0.05/J = 4$ holds 28/28, rate = 0.9911 exact, seed 42, $B = 200$, grid 112	`sochcontagion` (async)

Four methods (`ksg_te`, `ksg_robustness`, `namh_pipeline`, `soch_robustness`) route to the asynchronous tower; the remaining twenty-two are synchronous on the kernel. The figures above are reproduced from the structural reference `ARCHITECTURE.md` §A2 and carry the same provenance as Appendix D.

B API reference

The engine presents two HTTP surfaces: the synchronous kernel (`server/compute-server.mjs`, on Cloud Run) and the asynchronous tower (`server/job-server.mjs`, on the workstation, port 3030). Both speak the same data contract: one JSON object in, one JSON object out, with no path by which a caller supplies code rather than a method name and typed parameters. The static read surfaces only ever *read* these routes; they never compute.

Kernel routes (synchronous)

CORS is open for the static read-side pages. The two language-model routes share a daily budget; over cap they return `503 daily_capacity` with a reset hint, never a fabricated answer.

Route	Verb	Purpose	Cap
`/api/compute/run`	POST	run one registry method on schema-validated params	20/min
`/api/compute/catalog`	GET	the method + dataset registry the workbench binds to	–
`/api/chat`	POST	AI analyst: Gemini 2.5 function-calling over the registry	10/min, 50/day
`/api/research`	POST	deep-research assistant (agentic run + grounded synthesis)	5/min, 20/day
`/api/situate`	POST	place a result against the verified record	30/min
`/api/claims`	GET	epistemic ledger (established / contested / superseded)	–
`/api/sri/current`	GET	systemic-risk live-feed: latest value	–
`/api/sri/history`	GET	systemic-risk live-feed: series	–
`/api/sri/network`	GET	systemic-risk live-feed: current network	–
`/api/sri/cron-tick`	POST	the daily SRI append (Cloud Scheduler only)	–
`/api/event`	POST	read-side telemetry	60/min
`/health`	GET	liveness: methods, revision, sandbox, timeout	–
`/metrics`	GET	counters	–

The compute contract

A run names a method and a schema-validated parameter object. Schema validation rejects anything that is not a registered method or whose parameters fail their typed checks before any process is spawned.

```
POST /api/compute/run
{ "method": "ksg_te", "params": { "lag": 1, "k": 4 } }

200 OK
{ "method": "ksg_te",
  "result": { "top_edge": "USA->Japan", "te": 0.154234,
              "n_pairs": 306, "n_significant": 108 },
  "compute_path": "gpu", "elapsed_s": 2136.4,
  "data_sha256": "..." }
```

Listing 4: A synchronous method call and its single-JSON reply.

The reply carries a `compute_path` field (`gpu` or `cpu`) and the SHA-256 of the input panel, so a number always carries its data state. A method that exceeds `COMPUTE_TIMEOUT_S` (90 s in the deployed image) returns an explicit `timeout after Ns` error, never a silent hang.

Tower routes (asynchronous)

The four heavy methods (`ksg_te`, `ksg_robustness`, `namh_pipeline`, `soch_robustness`) are submitted to the tower, which returns a job id immediately and runs the work at a small fixed concurrency (`MAX_CONCURRENT` = 2). The contract is specified in `openapi/jobs.yaml`.

Route	Verb	Purpose
`/api/jobs/submit`	POST	workspace-gated submit; returns `job_<date>_<hash>`
`/api/jobs/:id`	GET	job status and, on completion, the result object
`/api/jobs/:id/stream`	GET	server-sent-events progress stream

Jobs persist to `.jobs/` on disk with a thirty-day time-to-live, so a result survives a restart of the tower process. Progress events are emitted by the R runners through `ce_progress()` only when the environment sets `CE_PROGRESS=1`, so the synchronous kernel's pure-single-JSON stdout is never polluted by progress lines. The kernel and the dashboard poll `/api/jobs/:id` for completion.

C The learning ledger

The engine keeps a staged record of operational lessons in `STATE.md`: a burned lesson enters at *fail*, *investigate* names the cause, *verify* proves the fix, and *distil* promotes it to a consultable skill that future sessions read before acting. This is the machinery by which the system learns from its own failures rather than repeating them. We reproduce the ledger below in condensed form; each row is a real incident with a named cause and a verified fix.

ID	Stage	Lesson
L1	distilled	A pre-registered robustness grid must anchor through the paper's own scripts and estimation universe before any badge is written: the package-default universe gave $p\,0.158$ where the published 56-pair design gives 0.042. Stop on anchor failure, document the amendment openly, keep the auxiliary run.
L2	distilled	`cloudrun/deploy.sh` resolves the API key from env first, then `$REPO/../.env.local`; moving the engine root silently ships a chat-disabled revision. Check the WARN line; restore with `gcloud run services update -update-env-vars` (no rebuild).
L3	distilled	Yahoo serves `close: null` for ten hours after a session; the finite-close guard makes the 06:00Z tick skip honestly and composition heals the gap next day. Never fire the SRI tick manually during the US session: the schedule *is* the partial-bar guard.
L4	distilled	Eval failures split into band failures (engine wrong: stop, investigate) and extraction failures (harness misread the result shape: fix the check, re-run the whole suite). Fixing extraction is calibration, not gaming, only while the expected band is untouched.
L5	distilled	"5-market IPS −77.26" was unreproducible until the market set was pinned to {India, USA, UK, China, Japan}; a different fifth market gives −80.16. Documented tuples must name their full parameterisation or they are not reproducible claims.

continued on next page

ID	Stage	Lesson
L6	distilled	LaTeX source grep is insufficient for count consistency: TikZ figure labels and line-wrapped phrases are invisible to line-based search but visible in the rendered PDF. Verify with `pdftotext file - | tr` newline `space | grep`, ligature-tolerant, across all figure sources.
L7	distilled	A laptop reboot killed both `setsid` daemons mid-pipeline; the eval suite survived because it had already written its artefact. Long-lived local services need a reboot-surviving re-arm; pipelines should write artefacts as they go, not at the end.
L8	distilled	Bibliography titles are looked up, never reconstructed from codenames: "MCPFM" resolves to a Model Context Protocol title (arXiv:2507.08065), not the codename expansion. Verify id ↔ title ↔ authors against the registered record before citing.
L9	verify	A dual-use ESM script (CLI + importable) must guard its CLI dispatch with the main-module check: a seeder's top-level dispatch ran the wrong selftest and `process.exit`'d, masquerading as a pass. Fixed and selftested.
L10	verify	Before writing code against a large vendored dependency, grep the dependency's own source for exact symbol names first: the Lean spectral proof compiled first-try with zero name-risk iterations because the mathlib symbols were verified from source pre-write. The code-API corollary of L8.
L11	verify	The 22-core hang had one named cause: three runners hard-coded `detectCores()-1` (= 21 workers), ignoring the job's `n_cores`; two concurrent jobs forked 2×21 workers over unpinned BLAS. Fix: one governed budget `ce_ncores()` with a server-set hard ceiling on all six fan-out sites, BLAS pinned to 1. Parallel fan-out must be governed by a ceiling, never by `detectCores()` under concurrency.
L12	verify	GPU offload is feasible with zero install: `numba` 0.63.1 + CUDA 12.4 are present and the RTX 3000 Ada is supported. The k-NN transfer-entropy search ports to two `numba.cuda` kernels, bit-exact against the FP64 CPU reference, 32–74× faster. The bar held: the GPU must equal the CPU, not approximate it.
L13	verify	`ksg_te` and `ksg_robustness` dispatch to the GPU with governed-CPU fallback (default AUTO; `CE_GPU=0` forces CPU). The bit-exactness gate caught a non-obvious boundary convention in the engine's own 1-D count, which the GPU then replicated host-side. Gate on the estimator's own output, including its floating-point boundary quirks, not the textbook form. Full-panel re-run: 35.6 min on one core; the 22-core saturation is gone.

The distilled rows (L1–L8) have been promoted to named skills that future sessions consult before acting; the verify-stage rows (L9–L13) carry a verified fix and are pending distillation. The ledger's own structural wiring (its steps exist, its artefact paths agree, the claims map is a subset of the suite rows, no nightly step deletes, the gap rule and its review gate remain intact) is itself a failable evaluation, `evals/loop-integrity.test.mjs`, which began at 11/20 red and now passes.

D Verified results and provenance

Every quantitative claim in this manuscript traces to one of two sources: a live computation against the engine's pre-registered band, or a documented published result reproduced by the engine. This appendix collects those figures with their provenance, so a reader can locate the origin of any number we state. Engine state at the time of writing: **26 methods**, live revision `shssm-compute-00036-llf` (confirmed 2026-06-15 by `GET /health` returning `methods:26`, `sandbox:"timeout"`, `timeout_s:90`); the most recent full suite passed **26/26**. The analysis panel `G20.xlsx` spans 18 markets, 2006-01-12 to 2026-03-18, giving 306 directed pairs, and is byte-hashed at startup.

Engine and infrastructure figures

Quantity	Verified value	Provenance
Registry size	26 methods	`/health`; `run-evals.mjs`
Eval suite	26/26 pass	most recent full run (2026-06-12)
Core ceiling	`CE_MAX_CORES`= 10	$\lfloor$(cpus − 2)/MAX_CONCURRENT$\rfloor$ on the 22-core workstation
GPU bit-exactness	$\max \lvert\Delta\rvert \leq 7 \times 10^{-16}$	`gpu/equiv_gate.R` (6.7×10^{-16} in the NAMH re-estimation)
GPU speedup (governed)	3.5×	12 pairs, $B = 99$: GPU 82 s (∼1 core) vs CPU 289 s (∼7 cores)
GPU full-panel re-run	35.6 min, 1 core, peak 388 MB	306 pairs, $B = 99$; observed TE bit-exact
AI cost ceiling	≤ 800 paid turns/day	400 turns/instance/day × 2 instances

Method figures (most recent suite)

Method	Verified value	Note
`unit_root`	India ADF −49.18; UK −52.64	live re-probe −49.1796
`garch`	persistence $\alpha + \beta = 0.991$	GARCH(1,1)
`dfa_hurst`	India $H = 0.542$	DFA
`namh_hurst`	Gold $H = 0.490$	band $[0.4889, 0.4909]$; `namh` package
`panel_unit_root`	IPS −77.26, LLC −51.79	tuple {India,USA,UK,China,Japan}; tuple-sensitive (Brazil fifth → −80.16)
`vecm`	rank 3 (trace 7265.97→1851.19)	India/USA/UK
`granger`	6 edges (USA out-degree 3)	India/USA/UK/China
`wavelet`	India $d1 = 47.07\%$	MODWT variance
`wavelet_coherence`	mean 0.249, peak $d6$ (0.523)	USA/India scale-resolved
`connectedness`	TCI 30.25%	short 15.05 / medium 11.56 / long 3.64
`spillover_rolling`	mean TCI 28.39 (45 windows)	range 10.5–46.3
`rolling_dcc`	India–USA mean ρ 0.23	0.2295; range 0.15–0.31
`network`	6 nodes, 18 edges, density 0.60	0.5667; 6 communities
`quantile_var`	USA top driver, net +0.70	+0.7006; $\tau = 0.05$
`var_irf`	max root 0.7053 (lag 7)	stable
`wqte`	USA→India $\tau.05 = 0.039$	0.0391; MODWT+quantreg reimplementation
`soch_profile`	USA→India 4-scale agg 0.0391	peak $d4$; published 5-scale agg 0.0426
`ksg_te`	USA→Japan TE 0.154234	306 pairs, 108 significant; GPU ≡ CPU
`ksg_robustness`	mean Spearman 0.7282 (min 0.581)	USA→Japan ranked first across all 8 (k, lag)

Method	Verified value	Note
`namh_te`	window-1 mean -0.223222	bit-exact to cached ($\Delta\, 4.92 \times 10^{-9}$)
`namh_reproduce`	max $\lvert\Delta\rvert$ 1.55×10^{-15}	FDR amber 0/552 by construction
`namh_pipeline`	window mean -0.22322165	band $[-0.2242, -0.2222]$; FDR 0/552 degenerate; seed 42; job `job_20260612_0f2f0fd5` (1222 s)
`channel_attribution`	GFC trade 27.9% [26.1, 29.8]	reproduced 0.000 pp; 8 episodes
`sri_daily`	first 0.00849 (306 pairs); 2026-06-05 0.00709 (272 pairs)	band $[0.00848, 0.00850]$
`soch_robustness`	pass-rate 0.9911 (111/112)	baseline $\tau = 0.05/J = 4$ holds 28/28; seed 42; $B = 200$

Programme-element figures (reported honestly, positives and negatives)

- **SOCH:** SOCH-A $p = 0.042$; SOCH-B 28/28; SOCH-C $p = 0.105$ (*not* significant, immutable). Lean: `sorry`-count 1/12; `prop:spectrum` closed ($\|H(\omega)\|^2 = S(\omega)$).
- **NAMH** (Frontiers Paper I, 17 pp, 28 assets 2001–2026): US broadcasts (out-strength 9/20 annual, 26/79 quarterly); Japan receives (in-strength 12/20, PageRank 10/20, Katz 8/20). Honest exception: quarterly Katz leader Argentina 22 edges over Japan 21. GPU $\equiv$ CPU 6.7×10^{-16}; verification 44/44.
- **contagion-channels:** GFC trade channel 27.9%, reproduced to 0.000 pp ([10]).
- **WaveQTE:** the 50.0%-versus-27.9% divergence resolved as a method-and-partition difference, not a defect; the engine `wqte` is an honest MODWT+quantreg reimplementation.
- **commodity:** Subprime most-fragmented, $|FC| = 0.290$, 5 communities (23 commodities, 1991–2025).
- **MCPFM** ([5]): v1 AUC 0.581 (DeLong $p = 0.030$) / 0.915 (US-COVID); a pre-registered v2 re-evaluation returns AUC 0.470 (Hansen SPA $p = 0.031$), reported openly. Reproduce status: honest pending.

The tuple-sensitivity of `panel_unit_root` (L5), the unseeded-surrogate non-reproducibility of transfer-entropy p-values, and the degenerate NAMH FDR network are not blemishes to be hidden; they are facts of the record, and the engine is built to report them as such.

References

[1] Viral V. Acharya, Lasse Heje Pedersen, Thomas Philippon, and Matthew Richardson. Measuring systemic risk. *The Review of Financial Studies*, 30(1):2–47, 2017. doi: 10.1093/rfs/hhw088.

[2] Tobias Adrian and Markus K. Brunnermeier. CoVaR. *American Economic Review*, 106(7): 1705–1741, 2016. doi: 10.1257/aer.20120555.

[3] Jozef Baruník and Tomáš Křehlík. Measuring the frequency dynamics of financial connectedness and systemic risk. *Journal of Financial Econometrics*, 16(2):271–296, 2018. doi: 10.1093/jjfinec/nby001.

[4] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57 (1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.

[5] Avishek Bhandari. Multi-scale network dynamics and systemic risk: A model context protocol approach to financial markets, 2025.

[6] Avishek Bhandari. Econstellar: An open-source ai-augmented research engine for computational financial econometrics, 2026.

[7] Avishek Bhandari and Ipsita Parida. Scale-ordered contagion: A spectral theory of heterogeneous information adaptation in financial networks, 2026.

[8] Avishek Bhandari and Ipsita Parida. sochcontagion: Scale-ordered contagion estimators and tests, 2026. R package version 0.1.0.

[9] Avishek Bhandari and Hitesh Kumar Sahu. namh: Estimators for the network adaptive market hypothesis, 2026. R package version 0.1.0.

[10] Avishek Bhandari, Ipsita Parida, and Hitesh Kumar Sahu. What drives contagion? identifying and attributing cross-border transmission mechanisms, 2026.

[11] Avishek Bhandari, Ipsita Parida, and Hitesh Kumar Sahu. contagionchannels: Identification and attribution of contagion channels, 2026. R package version 0.1.3.

[12] Carl Boettiger. An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: 10.1145/2723872.2723882.

[13] William A. Brock and Cars H. Hommes. Heterogeneous beliefs and routes to chaos in a simple asset pricing model. *Journal of Economic Dynamics and Control*, 22(8–9):1235–1274, 1998. doi: 10.1016/S0165-1889(98)00011-6.

[14] Christian Brownlees and Robert F. Engle. SRISK: A conditional capital shortfall measure of systemic risk. *The Review of Financial Studies*, 30(1):48–79, 2017. doi: 10.1093/rfs/hhw060.

[15] William G. Dewald, Jerry G. Thursby, and Richard G. Anderson. Replication in empirical economics: The journal of money, credit and banking project. *The American Economic Review*, 76(4):587–603, 1986. JSTOR 1804393.

[16] Francis X. Diebold and Kamil Yilmaz. Measuring financial asset return and volatility spillovers, with application to global equity markets. *The Economic Journal*, 119(534): 158–171, 2009. doi: 10.1111/j.1468-0297.2008.02208.x.

[17] Francis X. Diebold and Kamil Yilmaz. Better to give than to receive: Predictive directional measurement of volatility spillovers. *International Journal of Forecasting*, 28(1):57–66, 2012. doi: 10.1016/j.ijforecast.2011.02.006.

[18] Francis X. Diebold and Kamil Yilmaz. On the network topology of variance decompositions: Measuring the connectedness of financial firms. *Journal of Econometrics*, 182(1):119–134, 2014. doi: 10.1016/j.jeconom.2014.04.012.

[19] Thomas Dimpfl and Franziska Julia Peter. Using transfer entropy to measure information flows between financial markets. *Studies in Nonlinear Dynamics and Econometrics*, 17(1): 85–102, 2013. doi: 10.1515/snde-2012-0044.

[20] Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970. doi: 10.1111/j.1540-6261.1970.tb00518.x.

[21] Stefan Frenzel and Bernd Pompe. Partial mutual information for coupling analysis of multivariate time series. *Physical Review Letters*, 99(20):204101, 2007. doi: 10.1103/PhysRevLett.99.204101.

[22] Sanford J. Grossman and Joseph E. Stiglitz. On the impossibility of informationally efficient markets. *The American Economic Review*, 70(3):393–408, 1980.

[23] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.

[24] Roger Koenker and Achim Zeileis. On reproducible econometric research. *Journal of Applied Econometrics*, 24(5):833–847, 2009. doi: 10.1002/jae.1083.

[25] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004. doi: 10.1103/PhysRevE.69.066138.

[26] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, 2015. doi: 10.1145/2833157.2833162.

[27] Pierre L'Ecuyer. Good parameters and implementations for combined multiple recursive random number generators. *Operations Research*, 47(1):159–164, 1999. doi: 10.1287/opre.47.1.159.

[28] Andrew W. Lo. The adaptive markets hypothesis: Market efficiency from an evolutionary perspective. *Journal of Portfolio Management*, 30(5):15–29, 2004. doi: 10.3905/jpm.2004.442611.

[29] Robert Marschinski and Holger Kantz. Analysing the information flow between financial time series. *The European Physical Journal B*, 30(2):275–281, 2002. doi: 10.1140/epjb/e2002-00379-2.

[30] B. D. McCullough and H. D. Vinod. The numerical reliability of econometric software. *Journal of Economic Literature*, 37(2):633–665, 1999. doi: 10.1257/jel.37.2.633.

[31] C.-K. Peng, S. V. Buldyrev, S. Havlin, M. Simons, H. E. Stanley, and A. L. Goldberger. Mosaic organization of DNA nucleotides. *Physical Review E*, 49(2):1685–1689, 1994. doi: 10.1103/PhysRevE.49.1685.

[32] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

[33] Thomas Schreiber. Measuring information transfer. *Physical Review Letters*, 85(2):461–464, 2000. doi: 10.1103/PhysRevLett.85.461.

[34] Thomas Schreiber and Andreas Schmitz. Improved surrogate data for nonlinearity tests. *Physical Review Letters*, 77(4):635–638, 1996. doi: 10.1103/PhysRevLett.77.635.

[35] Thomas Schreiber and Andreas Schmitz. Surrogate time series. *Physica D: Nonlinear Phenomena*, 142(3–4):346–382, 2000. doi: 10.1016/S0167-2789(00)00043-9.

[36] V. A. Traag, L. Waltman, and N. J. van Eck. From louvain to leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9:5233, 2019. doi: 10.1038/s41598-019-41695-z.